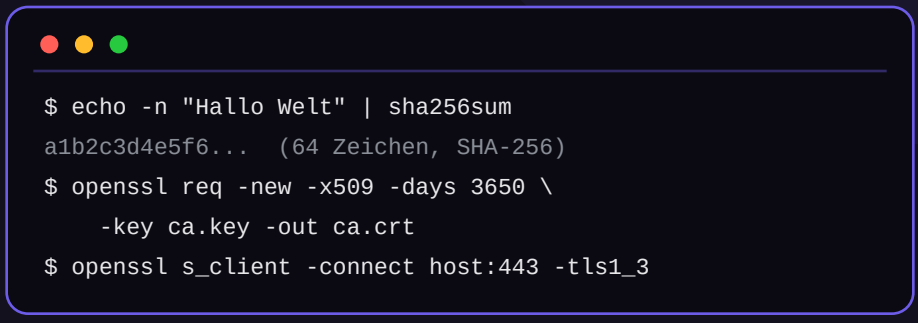
PRAXISHANDBUCH

Kryptographie verständlich

Von Hashes und Verschlüsselung bis TLS, Zertifikaten und digitaler Sicherheit



VERTRAUEN IST KEIN ZUFALL. SICHERHEIT IST MATHEMATIK.

Mit Übungen, Diagrammen, Spickzettel, Glossar und Index · Autor: Eugen Becker

Inhaltsverzeichnis

Vorwort 1

Warum Kryptographie heute unverzichtbar ist 1
Kryptographie im Alltag 1
Kryptographie in Unternehmen 1
Bedeutung für Netzwerkadministratoren 1
Bedeutung für Ethical Hacker und Blue Teams 1
Verbindung zu den vorhandenen Praxishandbüchern 2
Für wen ist dieses Handbuch? 2
Aufbau des Buches 2
Voraussetzungen für die Praxisübungen 3

Kapitel 1: Einführung – Was ist Kryptographie? 5

Was ist Kryptographie? 5
Warum wird Kryptographie benötigt? 5
Historische Entwicklung 5
Klassische Verschlüsselungsverfahren 6
Moderne Kryptographie 6
Die vier klassischen Sicherheitsziele 6
Unterschied zwischen Kodierung, Verschlüsselung und Hashing 7
Kryptographie im Alltag, in Betriebssystemen und in Netzwerken 8
Zusammenfassung 9
Kontrollfragen 9
Lernzielkontrolle 10

Kapitel 2: Grundlagen – Wie funktionieren Hashfunktionen? 11

Was ist eine Hashfunktion? 11
Eingabedaten, Hashwerte und Determinismus 11
Der Avalanche-Effekt 12
Kollisionsresistenz, Preimage- und Second-Preimage-Resistenz 12
MD5, SHA-1, SHA-256, SHA-512 und SHA-3 12
Typische Einsatzgebiete von Hashfunktionen 14
Praxisübungen 14
Zusammenfassung 16

Kontrollfragen 16
Lernzielkontrolle 17

Kapitel 3: Passwort-Hashes und sichere Passwortspeicherung 18

Wie werden Passwörter gespeichert? 18
Warum dürfen Passwörter nicht im Klartext gespeichert werden? 18
Unterschied zwischen Passwort und Passwort-Hash 18
Salt 19
Pepper 19
Rainbow Tables 19
Brute Force und Wörterbuchangriffe 20
Passwortentropie und Passwortkomplexität 20
Argon2id, bcrypt, scrypt und PBKDF2 20
Warum schnelle Hashfunktionen für Passwortspeicherung ungeeignet sind 20
Moderne Passwortspeicherung und typische Implementierungsfehler 21
Praxisübungen 21
Zusammenfassung 22
Kontrollfragen 23
Lernzielkontrolle 23

Kapitel 4: Symmetrische Verschlüsselung 25

Grundprinzip 25
Gemeinsamer geheimer Schlüssel 25
AES 25
Blockchiffren und Betriebsmodi 26
Initialisierungsvektor, Nonce und Authenticated Encryption 27
Schlüsselgenerierung und Schlüsselaufbewahrung 28
Typische Sicherheitsfehler 28
Praxisübungen 28
Zusammenfassung 31
Kontrollfragen 31
Lernzielkontrolle 32

Kapitel 5: Asymmetrische Verschlüsselung 33

Grundprinzip 33

Public Key, Private Key und Schlüsselpaar 33
RSA 34
Diffie-Hellman 34
Elliptic Curve Cryptography, ECC und ECDH 34
Schlüsselverteilung und Schlüsselgenerierung 35
Hybridverschlüsselung 35
Vorteile und Grenzen asymmetrischer Verfahren 36
Praxisübungen 36
Zusammenfassung 38
Kontrollfragen 38
Lernzielkontrolle 39

Kapitel 6: Digitale Signaturen 40
Was ist eine digitale Signatur? 40
Hashwert und Signatur 40
Signieren und Verifizieren 40
Integritätsprüfung und Authentizität 41
Unterschied zwischen Verschlüsselung und Signatur 41
RSA-Signaturen, ECDSA und Ed25519 41
Typische Einsatzgebiete 42
Technische Prüfung und rechtliche Verbindlichkeit 42
Praxisübungen 42
Zusammenfassung 43
Kontrollfragen 44
Lernzielkontrolle 44

Kapitel 7: Public Key Infrastructure und Zertifikate 46
Was ist eine PKI? 46
Certificate Authority, Root CA und Intermediate CA 46
X.509-Zertifikate 47
Certificate Signing Request 47
Zertifikatsausstellung und Zertifikatskette 47
Subject und Subject Alternative Name 47
Gültigkeitszeitraum, Zertifikatswiderruf, CRL und OCSP 47
Schlüsselverwaltung und Hardware Security Module 48
Typische Zertifikatsfehler 48

Praxisübungen 48
Zusammenfassung 51
Kontrollfragen 51
Lernzielkontrolle 52

Kapitel 8: TLS und HTTPS verständlich erklärt 53

Warum TLS benötigt wird 53
Unterschied zwischen SSL und TLS 53
Entwicklung der TLS-Versionen 53
Der TLS-1.3-Handshake 54
Schlüsselaustausch und Sitzungsschlüssel 54
Zertifikatsprüfung 55
Forward Secrecy 55
HTTPS und Cipher Suites 55
Typische TLS-Fehler und unsichere Konfigurationen 56
Praxisübungen 56
Zusammenfassung 58
Kontrollfragen 58
Lernzielkontrolle 59

Kapitel 9: Kryptographie in der Netzwerkkommunikation 60

Kryptographie und OSI-Modell 60
Verschlüsselung auf verschiedenen Netzwerkebenen 60
HTTPS, SSH, VPN und IPsec 60
WPA2 und WPA3 61
DNS over HTTPS und DNS over TLS 61
E-Mail-Verschlüsselung 61
Ende-zu-Ende-Verschlüsselung gegenüber Transportverschlüsselung 61
Verschlüsselung im Vergleich 62
Typische Netzwerkfehlkonfigurationen 62
Praxisübungen 62
Zusammenfassung 64
Kontrollfragen 64
Lernzielkontrolle 65

Kapitel 10: Kryptographie und Ethical Hacking – Hashcat und John the Ripper 66
Einordnung statt Wiederholung 66
Was ist Hashcat, was ist John the Ripper? 66
Hashidentifikation und unterstützte Hashformate 67
Passwort-Hashes untersuchen und offline Passwortprüfung 67
Wörterbuchangriffe, regelbasierte Angriffe und Maskenangriffe 68
GPU und CPU 68
Warum manche Hashverfahren leichter angreifbar sind 68
Einfluss von Salt, Iterationen und Rechenkosten 69
Argon2id und moderne Passwortspeicherung 69
Grenzen des Passwort-Crackings 69
Schutzmaßnahmen gegen Passwortangriffe 69
Praxisübungen 70
Zusammenfassung 71
Kontrollfragen 72
Lernzielkontrolle 73

Kapitel 11: Moderne kryptographische Herausforderungen 74
Quantencomputer als neue Bedrohung 74
Shors Algorithmus 74
Grovers Algorithmus 74
Post-Quantum Cryptography 75
ML-KEM, ML-DSA und SLH-DSA 75
NIST-Standardisierung 76
Kryptographische Agilität 76
Zukunft der Verschlüsselung und Migration bestehender Systeme 76
Zusammenfassung 76
Kontrollfragen 77
Lernzielkontrolle 77

Kapitel 12: Kryptographische Fehler und Sicherheitsrisiken 79
Eine Zusammenschau statt neuer Theorie 79
Schwache Schlüssel und unsichere Zufallszahlengeneratoren 79
Wiederverwendung von Nonces 79

Veraltete Algorithmen und unsichere Hashverfahren 80
Zertifikatsfehler, Man-in-the-Middle und Downgrade-Angriffe 80
Fehlkonfigurationen und schlechte Schlüsselverwaltung 80
Unsichere Implementierungen 80
Übersicht: Fehler und ihre Gegenmaßnahme 80
Typische Fehlannahmen 81
Maßnahmen zur Absicherung 81
Zusammenfassung 82
Kontrollfragen 82
Lernzielkontrolle 83

Kapitel 13: Praxisprojekt – Eine sichere kryptographische Testumgebung 84

Projektziel 84
Laborumgebung 84
Aufgabe 1: SHA-256-Hashes erzeugen und vergleichen 85
Aufgabe 2: Eine Datei mit AES verschlüsseln 85
Aufgabe 3: Ein RSA-Schlüsselpaar erstellen 86
Aufgabe 4: Eine digitale Signatur erzeugen und prüfen 87
Aufgabe 5: Eine eigene Test-CA einrichten 87
Aufgabe 6: Ein TLS-Testzertifikat erstellen 88
Aufgabe 7: Eine HTTPS-Verbindung untersuchen 89
Aufgabe 8: TLS-Verkehr mit Wireshark analysieren 90
Aufgabe 9: Einen eigenen Passwort-Hash untersuchen 90
Aufgabe 10: Unterschiedliche Passwort-Hashverfahren vergleichen 90
Aufgabe 11: Ergebnisse dokumentieren 91
Aufgabe 12: Eine Sicherheitsbewertung erstellen 91
Zusammenfassung 92

Kapitel 14: Zusammenfassung und Wissenstest 93

Die wichtigsten kryptographischen Verfahren im Überblick 93
Entscheidungshilfe: Welches Verfahren für welchen Zweck? 94
Multiple-Choice-Fragen 94
Praktische Verständnisaufgaben 95
Lösungen mit ausführlichen Erklärungen 95
Zusammenfassung 96

Anhang **97**

OpenSSL-Spickzettel 97
Hashcat-Spickzettel 98
John-the-Ripper-Spickzettel 98
Linux-Befehle dieses Buches 98
Windows-PowerShell-Befehle dieses Buches 99
Wireshark-Filter für TLS 99
Übersicht kryptographischer Algorithmen 99
Post-Quantum-Cryptography im Überblick 100
Glossar 100
Weiterführende Ressourcen 101
Quellenverzeichnis 102
Verzeichnis der Befehlsbeispiele 103
Index 104

Vorwort

Kaum ein anderes Teilgebiet der IT ist so unterschätzt wie Kryptographie, obwohl sie praktisch jede digitale Handlung absichert: eine Überweisung, eine WhatsApp-Nachricht, ein Login, ein Software-Update. Dieses Handbuch erklärt, wie das im Einzelnen funktioniert, ohne ein abgeschlossenes Mathematikstudium vorauszusetzen.

Warum Kryptographie heute unverzichtbar ist

Ohne Kryptographie gäbe es kein vertrauenswürdiges Internet. Jede HTTPS-Verbindung, jede Banking-App, jedes gespeicherte Passwort hängt von denselben mathematischen Grundprinzipien ab, die in diesem Buch Schritt für Schritt aufgebaut werden.

Kryptographie im Alltag

Ein Vorhängeschloss-Symbol im Browser, eine Chatnachricht, die nur der Empfänger lesen kann, eine Banking-App, die ein Fingerabdruck-Login akzeptiert: Hinter jeder dieser alltäglichen Situationen steckt eines der in diesem Buch behandelten Verfahren.

Kryptographie in Unternehmen

Unternehmen verschlüsseln Datenbanken, signieren Software-Updates, betreiben eigene Zertifizierungsstellen für interne Dienste und schützen Backups. Wer diese Entscheidungen versteht statt nur Haken in einer Checkliste zu setzen, trifft bessere Entscheidungen.

Bedeutung für Netzwerkadministratoren

Wer TLS-Zertifikate verwaltet, VPNs konfiguriert oder WLAN-Sicherheit einrichtet, trifft laufend kryptographische Entscheidungen: Welches Verfahren, welche Schlüssellänge, welche Gültigkeitsdauer. Dieses Buch liefert die Grundlage dafür.

Bedeutung für Ethical Hacker und Blue Teams

Wer Passwort-Hashes mit Hashcat oder John the Ripper prüft, muss verstehen, warum manche Hashverfahren in Sekunden fallen und andere Jahre widerstehen. Genau diese Grundlage legt Kapitel 10.

Verbindung zu den vorhandenen Praxishandbüchern

Dieses Buch baut nicht auf einem bestimmten Band einer Reihe auf, sondern ergänzt die bereits vorhandenen Praxishandbücher um das gemeinsame mathematische Fundament, auf dem viele von ihnen aufsetzen: John the Ripper und Hashcat nutzen es beim Passwort-Cracking, Wireshark macht es beim Betrachten von TLS-Verkehr sichtbar, die Windows- und Linux-Security-Reihen setzen es bei BitLocker, SSH-Schlüsseln und Zertifikaten voraus.

Für wen ist dieses Handbuch?

Du bist hier richtig, wenn du

- ohne Vorkenntnisse verstehen willst, was hinter dem Schloss-Symbol im Browser steckt,
- dich auf eine Ausbildung zum Fachinformatiker für Systemintegration vorbereitest,
- als Systemadministrator Zertifikate und TLS-Verbindungen nicht nur konfigurierst, sondern wirklich verstehst,
- als Ethical Hacker oder Blue-Team-Analyst einordnen willst, warum Hashcat manche Passwörter in Sekunden findet und andere nicht.

Aufbau des Buches

Kapitel	Thema
1	Einführung: Was ist Kryptographie?
2	Grundlagen: Wie funktionieren Hashfunktionen?

Kapitel	Thema
3	Passwort-Hashes und sichere Passwortspeicherung
4	Symmetrische Verschlüsselung
5	Asymmetrische Verschlüsselung
6	Digitale Signaturen
7	Public Key Infrastructure und Zertifikate
8	TLS und HTTPS verständlich erklärt
9	Kryptographie in der Netzwerkkommunikation
10	Kryptographie und Ethical Hacking: Hashcat und John the Ripper
11	Moderne kryptographische Herausforderungen
12	Kryptographische Fehler und Sicherheitsrisiken
13	Praxisprojekt: Eine sichere kryptographische Testumgebung
14	Zusammenfassung und Wissenstest

Voraussetzungen für die Praxisübungen

Für die Übungen in diesem Buch genügt eine einzelne Linux-VM (Debian, Ubuntu oder Kali) mit installiertem OpenSSL und Python 3 samt dem Paket `cryptography` (`pip install cryptography`), idealerweise ergänzt um eine zweite Maschine oder einen Windows-Client für die netzwerkbezogenen Übungen in Kapitel 8 und 9. Hashcat und John the Ripper werden in Kapitel 10 benötigt und können parallel auf derselben VM installiert werden.

Gut zu wissen

OpenSSL ist auf den meisten Linux-Distributionen bereits vorinstalliert. Prüfe die vorhandene Version mit `openssl version` und aktualisiere bei Bedarf über die Paketverwaltung deiner Distribution. Das Python-Paket `cryptography` wird in Kapitel 4 für eine Übung benötigt, die `openssl enc` bewusst nicht verwendet, da dieses Werkzeug keine authentifizierten AES-GCM-Modi unterstützt.

Kapitel 1: Einführung – Was ist Kryptographie?

Lernziele

- Du kannst Kryptographie und Kryptologie unterscheiden.
- Du ordnest ein Beispiel korrekt einem der vier klassischen Sicherheitsziele zu.
- Du unterscheidest Kodierung, Verschlüsselung und Hashing an einem Beispiel.

Was ist Kryptographie?

Kryptographie ist die Lehre davon, Informationen so zu verändern, dass nur berechtigte Personen sie wieder in ihre ursprüngliche, lesbare Form zurückverwandeln können, oder so, dass sich nachträglich prüfen lässt, ob sie unverändert geblieben sind. Das Wort stammt aus dem Griechischen: *kryptós* (verborgen) und *gráphein* (schreiben).

Kryptologie ist der umfassendere Begriff und schließt neben der Kryptographie auch die **Kryptoanalyse** ein, also die Lehre davon, wie sich verschlüsselte oder gehashte Daten ohne den eigentlich vorgesehenen Schlüssel brechen lassen. Kryptographie baut, Kryptoanalyse bricht; beide zusammen bilden die Kryptologie.

Warum wird Kryptographie benötigt?

Ohne Kryptographie könnte jeder, der eine Netzwerkverbindung mitliest (vergleiche das Wireshark-Handbuch dieser Fachbibliothek), jede übertragene Information im Klartext lesen. Kryptographie macht aus einer offenen Leitung eine vertrauliche, auch wenn die Leitung selbst technisch weiterhin offen bleibt.

Historische Entwicklung

Die **Caesar-Chiffre** der Antike verschob jeden Buchstaben um eine feste Anzahl Stellen im Alphabet, etwa B wird zu E bei einer Verschiebung um

drei. Sie lässt sich mit nur 25 Versuchen vollständig brechen. Die **Vigenère-Chiffre** der Renaissance nutzte ein wiederholtes Schlüsselwort und galt lange als unknackbar, bis im 19. Jahrhundert statistische Methoden sie brachen. Die **Enigma** im Zweiten Weltkrieg war eine mechanische Rotor-Chiffriermaschine; ihr Bruch durch das Team um Alan Turing in Bletchley Park gilt als einer der Wendepunkte der Kryptoanalyse. Moderne Kryptographie seit den 1970er-Jahren verzichtet auf geheime Verfahren und setzt stattdessen auf öffentlich bekannte, mathematisch geprüfte Algorithmen, deren Sicherheit allein vom geheim gehaltenen Schlüssel abhängt – ein Prinzip, das als **Kerckhoffs' Prinzip** bekannt ist.

Beispiel 1.1: Caesar-Chiffre von Hand nachvollziehen (Verschiebung 3)

```
Klartext:  ANGRIFF
Geheimtext: DQJULII
```

Klassische Verschlüsselungsverfahren

Caesar- und Vigenère-Chiffre gehören zu den **monoalphabetischen** beziehungsweise **polyalphabetischen** Substitutionsverfahren: Buchstaben werden gegen andere Buchstaben ausgetauscht, nach einer festen oder zyklisch wechselnden Regel. Beide sind aus heutiger Sicht ausschließlich von historischem Interesse.

Moderne Kryptographie

Moderne Verfahren arbeiten nicht mit Buchstaben, sondern mit Bits und nutzen mathematische Probleme, die mit heutiger Rechenleistung praktisch nicht umkehrbar sind, etwa das Faktorisieren sehr großer Zahlen (Kapitel 5) oder das diskrete Logarithmusproblem auf elliptischen Kurven. Band 4 bis 9 dieses Buches bauen genau diese modernen Verfahren systematisch auf.

Die vier klassischen Sicherheitsziele

Ziel	Bedeutung	Beispiel einer Verletzung
Vertraulichkeit	nur berechtigte Personen können die Information lesen	ein mitgelesenes, unverschlüsseltes Passwort
Integrität	die Information bleibt unverändert, Änderungen sind erkennbar	eine unbemerkt manipulierte Überweisung
Authentizität	die Herkunft der Information ist eindeutig und nachprüfbar	eine gefälschte Absenderadresse
Verbindlichkeit	der Urheber kann seine Urheberschaft nicht im Nachhinein abstreiten	ein signierter Vertrag, dessen Unterzeichner die Signatur leugnet

> **Gut zu wissen**
>
> Verbindlichkeit wird im Englischen als *Non-Repudiation* bezeichnet. Eine digitale Signatur (Kapitel 6) ist das zentrale technische Werkzeug dafür, ersetzt aber nicht automatisch die rechtliche Bewertung: Eine gültige Signatur beweist, dass die Daten zu einem bestimmten Schlüsselpaar passen und seit der Signierung unverändert sind, nicht automatisch, welche natürliche Person den privaten Schlüssel tatsächlich verwendet hat oder welche rechtliche Wirkung das im Einzelfall hat. Das vertieft Kapitel 6.

Unterschied zwischen Kodierung, Verschlüsselung und Hashing

Diese drei Begriffe werden in der Praxis häufig verwechselt, obwohl sie grundverschiedene Zwecke erfüllen.

Beispiel 1.2: Dieselbe Information, drei unterschiedliche Operationen (Konzept)

```
Kodierung:      "Hallo" -> Base64 -> "SGFsbG8=" -> zurück zu
    "Hallo"
```

```
                (jeder kann das ohne Schlüssel umkehren, dient nur
                    der
                 Darstellung in einem bestimmten Format)

Verschlüsselung: "Hallo" -> AES mit Schlüssel K -> Geheimtext
                -> zurück zu "Hallo" (nur mit Schlüssel K möglich)

Hashing:        "Hallo" -> SHA-256 -> ein fester Hashwert
                (nicht zum Zurückrechnen vorgesehen, siehe Hinweis
                    unten)
```

Kodierung (zum Beispiel Base64) wandelt Daten in ein anderes, oft für die Übertragung besser geeignetes Format um, ohne jeden Schutzanspruch; jeder kann sie ohne Schlüssel umkehren. **Verschlüsselung** macht Daten mit einem Schlüssel unlesbar und mit demselben oder einem passenden Schlüssel wieder lesbar (Kapitel 4 und 5). **Hashing** erzeugt aus beliebig langen Eingabedaten einen Fingerabdruck fester Länge, der nicht dafür ausgelegt ist, die Eingabe zurückzuliefern (Kapitel 2).

> **Gut zu wissen**
>
> „Nicht zurückrechnen" beschreibt die Hashfunktion selbst, nicht jede Eingabe gleichermaßen. Bei einer schwachen oder bekannten Eingabe, etwa einem gängigen Passwort, kann ein Angreifer trotzdem erfolgreich sein, indem er Kandidaten hasht und mit dem Zielwert vergleicht, statt den Hashwert mathematisch umzukehren. Kapitel 3 und 10 vertiefen genau diesen Unterschied.

> **WARNUNG – Sicherheitsrisiko**
>
> Base64 wird gelegentlich fälschlich als „Verschlüsselung" bezeichnet. Es handelt sich um eine reine Kodierung ohne jeden Schlüssel und bietet keinerlei Schutz der Vertraulichkeit.

Kryptographie im Alltag, in Betriebssystemen und in Netzwerken

Im Alltag begegnet dir Kryptographie beim Online-Banking, beim Messenger und beim kontaktlosen Bezahlen. In Betriebssystemen schützt sie

gespeicherte Passwörter (Kapitel 3) und ganze Festplatten, etwa über BitLocker unter Windows oder LUKS unter Linux, beide ausführlich in den jeweiligen Security-Handbüchern dieser Fachbibliothek behandelt. In Netzwerken sichert sie praktisch jede Verbindung ab, von SSH über VPN bis HTTPS (Kapitel 9).

Zusammenfassung

> **Zusammenfassung**
>
> • Kryptographie verändert Informationen gezielt; Kryptoanalyse versucht, das ohne Schlüssel rückgängig zu machen.
> • Die vier klassischen Sicherheitsziele sind Vertraulichkeit, Integrität, Authentizität und Verbindlichkeit.
> • Kodierung, Verschlüsselung und Hashing erfüllen unterschiedliche Zwecke und dürfen nicht verwechselt werden.
> • Moderne Kryptographie beruht auf öffentlich bekannten Algorithmen, deren Sicherheit allein vom geheimen Schlüssel abhängt.

Kontrollfragen

1. Was unterscheidet Kryptographie von Kryptologie? (leicht)
2. Nenne die vier klassischen Sicherheitsziele. (leicht)
3. Warum gilt Base64 nicht als Verschlüsselung? (mittel)
4. Was besagt Kerckhoffs' Prinzip? (mittel)
5. Eine Nachricht wurde signiert, aber nicht verschlüsselt. Welches Sicherheitsziel ist dadurch erfüllt, welches nicht zwangsläufig? (schwer)

Lösungshinweise: 1. Kryptographie baut Verfahren, Kryptologie umfasst zusätzlich die Kryptoanalyse, die sie bricht. 2. Vertraulichkeit, Integrität, Authentizität, Verbindlichkeit. 3. Weil sie ohne jeden Schlüssel von jedem umkehrbar ist und keinen Schutz bietet. 4. Dass die Sicherheit eines Verfahrens allein vom geheimen Schlüssel abhängen soll, nicht von der Geheimhaltung des Verfahrens selbst. 5. Authentizität und Integrität sind erfüllt, da die Signatur Herkunft und Unveränderheit bestätigt; Vertraulichkeit ist nicht zwangsläufig erfüllt, da eine Signatur allein den Inhalt nicht verbirgt.

Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] Kryptographie und Kryptologie korrekt unterscheiden kannst,
- [ ] ein eigenes Beispiel für jedes der vier Sicherheitsziele formulieren kannst,
- [ ] Kodierung, Verschlüsselung und Hashing an einem selbst gewählten Beispiel auseinanderhalten kannst,
- [ ] erklären kannst, warum moderne Kryptographie auf geheimen Verfahren verzichtet.

Kapitel 2: Grundlagen – Wie funktionieren Hashfunktionen?

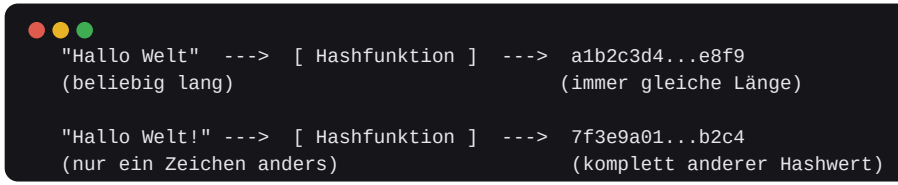
Lernziele

- Du erklärst die vier Grundeigenschaften einer kryptographischen Hashfunktion.
- Du berechnest Hashwerte unter Linux und Windows und vergleichst sie.
- Du erkennst anhand eines Hashwerts eine Dateiveränderung.

Was ist eine Hashfunktion?

Eine **Hashfunktion** nimmt eine beliebig lange Eingabe entgegen, etwa ein einzelnes Wort oder eine ganze Festplatte, und erzeugt daraus einen **Hashwert** fester Länge, oft als Folge von Hexadezimalzeichen dargestellt. Man kann sich das wie einen digitalen Fingerabdruck vorstellen: Zwei unterschiedliche Personen haben unterschiedliche Fingerabdrücke, und aus dem Fingerabdruck allein lässt sich die Person nicht körperlich rekonstruieren.

Diagramm 2.1: Grundprinzip einer Hashfunktion



Eingabedaten, Hashwerte und Determinismus

Eine Hashfunktion ist **deterministisch**: Dieselbe Eingabe ergibt immer denselben Hashwert, auf jedem System, zu jedem Zeitpunkt. Die Ausgabe hat außerdem eine **feste Länge**, unabhängig von der Länge der Eingabe: SHA-256 liefert immer 256 Bit, dargestellt als 64 Hexadezimalzeichen, ob die Eingabe ein einzelnes Zeichen oder eine ganze DVD ist.

Der Avalanche-Effekt

Der **Avalanche-Effekt** (Lawineneffekt) bedeutet, dass eine minimale Änderung der Eingabe, und sei es nur ein einziges Bit, den Hashwert vollständig und unvorhersehbar verändert, wie in Diagramm 2.1 gezeigt. Das verhindert, dass sich aus kleinen Eingabeänderungen irgendein Muster im Hashwert ableiten ließe.

Kollisionsresistenz, Preimage- und Second-Preimage-Resistenz

Eine sichere Hashfunktion muss drei Eigenschaften erfüllen, die sich auf unterschiedliche Angriffsrichtungen beziehen.

Eigenschaft	Bedeutet, dass ...
Kollisionsresistenz	es praktisch unmöglich ist, zwei beliebige Eingaben mit demselben Hashwert zu finden
Preimage-Resistenz	es praktisch unmöglich ist, aus einem Hashwert die ursprüngliche Eingabe zu rekonstruieren
Second-Preimage-Resistenz	es praktisch unmöglich ist, zu einer gegebenen Eingabe eine zweite mit demselben Hashwert zu finden

> **Gut zu wissen**
>
> „Praktisch unmöglich" bedeutet nicht mathematisch ausgeschlossen. Da nur endlich viele Hashwerte existieren, aber unendlich viele mögliche Eingaben, müssen Kollisionen theoretisch vorkommen (das sogenannte Taubenschlagprinzip). Ein sicheres Verfahren macht es nur praktisch, mit heutiger Rechenleistung in sinnvoller Zeit, unmöglich, eine solche Kollision gezielt zu finden.

MD5, SHA-1, SHA-256, SHA-512 und SHA-3

Verfahren	Ausgabelänge	Aktueller Status
MD5	128 Bit	gebrochen, Kollisionen praktisch erzeugbar; nicht sicherheitskritisch einsetzen
SHA-1	160 Bit	gebrochen (2017 erste praktische Kollision veröffentlicht); nicht sicherheitskritisch einsetzen
SHA-256	256 Bit	Teil der SHA-2-Familie, Stand der Technik, breit eingesetzt
SHA-512	512 Bit	Teil der SHA-2-Familie, auf manchen 64-Bit-Plattformen schneller als SHA-256, abhängig von Implementierung und Hardware
SHA-3	wählbar (224–512 Bit)	neuere, strukturell andere Familie (Keccak), als Alternative zu SHA-2 standardisiert

Die US-Standardisierungsbehörde NIST hat SHA-1 für neue, sicherheitskritische Anwendungen bereits seit Jahren nicht mehr empfohlen; ab 2030 gilt es laut aktuellem NIST-Übergangsplan als vollständig zu vermeiden, vorbehaltlich einer Bestätigung durch die jeweils aktuelle NIST-Veröffentlichung. SHA-256 und SHA-512 sind in FIPS 180-4 spezifiziert, SHA-3 in FIPS 202.

> **WARNUNG – Sicherheitsrisiko**
>
> MD5 und SHA-1 dürfen in diesem Buch zu Lernzwecken erklärt und in Übungen gezeigt werden, gehören aber in keine produktive, sicherheitskritische Anwendung mehr. Verwende für neue Projekte ausschließlich SHA-256, SHA-512 oder SHA-3.

Typische Einsatzgebiete von Hashfunktionen

Hashfunktionen prüfen Dateiintegrität nach einem Download, erkennen Duplikate, bilden die Grundlage von Passwort-Hashes (Kapitel 3) und digitalen Signaturen (Kapitel 6), und werden in Blockchain-Systemen zur Verkettung von Blöcken genutzt.

Praxisübungen

Übung A: Hashwerte unter Linux erzeugen

Lernziel: SHA-256-Hashwerte mit Bordmitteln berechnen.
Voraussetzungen: eine Linux-VM mit Standard-Coreutils.

Befehle zu Übung A

```
$ echo -n "Hallo Welt" | sha256sum
$ echo -n "Hallo Welt!" | sha256sum
```

Erwartetes Ergebnis: Zwei völlig unterschiedliche, 64-stellige Hexadezimalwerte, obwohl sich die Eingabe nur um ein Ausrufezeichen unterscheidet. **Erklärung:** `-n` bei `echo` unterdrückt den abschließenden Zeilenumbruch, damit genau der eingegebene Text gehasht wird, nicht der Text plus ein unsichtbares Zeichen. **Troubleshooting:** Erhältst du bei wiederholter Eingabe denselben Hashwert wie hier gezeigt nicht? Prüfe, ob `-n` tatsächlich gesetzt wurde; ohne diese Option würde ein zusätzliches Newline-Zeichen den Hashwert verändern.

Übung B: Hashwerte unter Windows mit PowerShell erzeugen

Lernziel: denselben Hashwert plattformübergreifend nachvollziehen.
Voraussetzungen: Windows mit PowerShell.

Befehle zu Übung B

```
PS> $bytes = [System.Text.Encoding]::UTF8.GetBytes("Hallo Welt")
PS> $stream = [System.IO.MemoryStream]::new($bytes)
PS> Get-FileHash -InputStream $stream -Algorithm SHA256
PS> $stream.Dispose()
```

Erwartetes Ergebnis: Derselbe Hashwert wie unter Übung A, sofern die Textkodierung übereinstimmt. **Erklärung:** `Get-FileHash` erwartet laut Microsoft-Dokumentation einen Dateipfad oder, über den Parameter `-InputStream`, einen Stream – kein rohes Byte-Array direkt aus der Pipeline. Der Text wird deshalb zunächst in einen `MemoryStream` verpackt, der sich wie eine Datei im Arbeitsspeicher verhält, und dieser anschließend ausdrücklich über `-InputStream` übergeben. **Troubleshooting:** Unterschiedlicher Hashwert trotz gleichem Text? Prüfe die Zeichenkodierung; UTF-8 und UTF-16 erzeugen für denselben sichtbaren Text unterschiedliche Bytefolgen und damit unterschiedliche Hashwerte. `$stream.Dispose()` gibt den belegten Arbeitsspeicher wieder frei und sollte nach jedem Testlauf ausgeführt werden.

Übung C: Dateiveränderungen erkennen

Lernziel: eine Dateimanipulation anhand des Hashwerts nachweisen. **Voraussetzungen:** wie Übung A.

Befehle zu Übung C

```
$ echo "Originalinhalt" > test.txt
$ sha256sum test.txt
$ echo "zusätzliche Zeile" >> test.txt
$ sha256sum test.txt
```

Erwartetes Ergebnis: Zwei unterschiedliche Hashwerte vor und nach der Änderung. **Erklärung:** Der Avalanche-Effekt sorgt dafür, dass bereits eine einzelne angehängte Zeile den gesamten Hashwert verändert, unabhängig davon, wie klein die Änderung war. **Troubleshooting:** Identischer Hashwert trotz Änderung? Prüfe, ob der `>>`-Operator (anhängen) statt versehentlich `>` (überschreiben mit identischem Inhalt) verwendet wurde.

Übung D: Hashverfahren vergleichen

Lernziel: die unterschiedliche Ausgabelänge verschiedener Verfahren selbst sehen. **Voraussetzungen:** wie Übung A.

Befehle zu Übung D

```
$ echo -n "Testtext" | md5sum
$ echo -n "Testtext" | sha1sum
$ echo -n "Testtext" | sha256sum
$ echo -n "Testtext" | sha512sum
```

Erwartetes Ergebnis: Vier unterschiedlich lange Hexadezimalwerte: 32, 40, 64 und 128 Zeichen. **Erklärung:** Jedes Hexadezimalzeichen kodiert 4 Bit; 32 Zeichen entsprechen 128 Bit (MD5), 128 Zeichen entsprechen 512 Bit (SHA-512). **Troubleshooting:** `md5sum` oder `sha1sum` nicht gefunden? Diese Werkzeuge sind auf manchen minimalen Linux-Installationen nicht vorinstalliert; `apt install coreutils` stellt sie nach.

Zusammenfassung

> **Zusammenfassung**
>
> • Eine Hashfunktion erzeugt aus beliebig langer Eingabe einen deterministischen Hashwert fester Länge.
> • Der Avalanche-Effekt sorgt dafür, dass kleinste Eingabeänderungen den gesamten Hashwert verändern.
> • Kollisions-, Preimage- und Second-Preimage-Resistenz sind die drei zentralen Sicherheitseigenschaften.
> • MD5 und SHA-1 gelten als gebrochen; SHA-256, SHA-512 und SHA-3 sind der aktuelle Stand der Technik.

Kontrollfragen

1. Was bedeutet „deterministisch" im Zusammenhang mit Hashfunktionen? (leicht)
2. Wie lang ist ein SHA-256-Hashwert in Hexadezimalzeichen? (leicht)
3. Was besagt der Avalanche-Effekt? (mittel)
4. Warum gilt SHA-1 heute als unsicher? (mittel)

5. Warum ist es laut Taubenschlagprinzip unmöglich, dass eine Hashfunktion vollständig kollisionsfrei ist, und warum ist das in der Praxis trotzdem kein Problem? (schwer)

Lösungshinweise: 1. Dieselbe Eingabe ergibt immer denselben Hashwert. 2. 64 Zeichen. 3. Dass eine minimale Eingabeänderung den gesamten Hashwert unvorhersehbar verändert. 4. Weil 2017 eine praktisch erzeugte Kollision veröffentlicht wurde. 5. Da es unendlich viele mögliche Eingaben, aber nur endlich viele Hashwerte gibt, müssen rein rechnerisch Kollisionen existieren; in der Praxis ist das unproblematisch, solange es niemandem gelingt, eine solche Kollision mit vertretbarem Rechenaufwand gezielt zu finden.

Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] die vier Grundeigenschaften einer Hashfunktion in eigenen Worten erklären kannst,
- [ ] SHA-256-Hashwerte unter Linux und Windows berechnet und verglichen hast,
- [ ] eine Dateiveränderung anhand eines veränderten Hashwerts nachgewiesen hast,
- [ ] begründen kannst, warum MD5 und SHA-1 nicht mehr sicherheitskritisch eingesetzt werden sollten.

Kapitel 3: Passwort-Hashes und sichere Passwortspeicherung

Lernziele

- Du erklärst, warum Passwörter gehasht statt verschlüsselt gespeichert werden.
- Du verstehst die Rolle von Salt und grenzt ihn von Pepper ab.
- Du erzeugst eigene Passwort-Hashes mit einem modernen Verfahren und vergleichst die Rechenzeit.

Wie werden Passwörter gespeichert?

Wenn du dich irgendwo anmeldest, vergleicht das System nicht dein eingegebenes Passwort mit einer gespeicherten Kopie. Es berechnet aus deiner Eingabe einen Hashwert (Kapitel 2) und vergleicht diesen mit einem zuvor gespeicherten Hashwert. Stimmen beide überein, war die Eingabe korrekt, ohne dass das System dein Passwort selbst jemals dauerhaft gespeichert hätte.

Warum dürfen Passwörter nicht im Klartext gespeichert werden?

Eine Datenbank mit Klartext-Passwörtern ist bei jedem Datenleck sofort vollständig kompromittiert. Eine Datenbank mit guten Passwort-Hashes zwingt einen Angreifer stattdessen, jeden einzelnen Hash mühsam zu erraten, mit stark unterschiedlichem Erfolg je nach Qualität des verwendeten Hashverfahrens, wie dieses Kapitel zeigt.

Unterschied zwischen Passwort und Passwort-Hash

Das **Passwort** ist das, was ein Mensch sich merkt und eintippt. Der **Passwort-Hash** ist das, was tatsächlich gespeichert wird: eine Einwegfunktion des Passworts, die nicht dafür ausgelegt ist, das Passwort zurückzuliefern (Preimage-Resistenz, Kapitel 2). Das schützt vor allem gegen eine direkte, mathematische Umkehrung des Hashwerts; ein

schwaches oder weit verbreitetes Passwort kann trotzdem gefunden werden, wenn ein Angreifer einfach genügend Kandidaten hasht und mit dem gestohlenen Wert vergleicht, genau das Verfahren, das Kapitel 10 anhand von Hashcat und John the Ripper vertieft.

Salt

Diagramm 3.1: Salting beim Passwort-Hashing

```
Ohne Salt:
  "passwort123" -> Hash -> gleicher Hash für jeden, der dieses Passwort nutzt

Mit Salt:
  "passwort123" + Salt "a8f3" -> Hash -> Hash A  (Nutzer 1)
  "passwort123" + Salt "7e21" -> Hash -> Hash B  (Nutzer 2)
  (gleiches Passwort, unterschiedlicher Salt, unterschiedlicher Hash)
```

Ein **Salt** ist ein zufälliger, pro Konto einzigartiger Wert, der vor dem Hashen an das Passwort angehängt wird. Dadurch ergeben zwei Nutzer mit demselben Passwort unterschiedliche Hashwerte, wie in Diagramm 3.1 gezeigt. Der Salt selbst ist kein Geheimnis und wird zusammen mit dem Hash gespeichert, damit das System ihn beim nächsten Login erneut anhängen kann.

Pepper

Ein **Pepper** ist, anders als der Salt, ein einzelnes, für die gesamte Anwendung geltendes Geheimnis, das zusätzlich zum individuellen Salt verwendet und getrennt von der Passwort-Datenbank aufbewahrt wird, etwa in einer Umgebungsvariable oder einem Schlüsseltresor. Ein gestohlener Datenbankauszug allein reicht dann nicht mehr aus, um die Hashes offline zu testen.

Rainbow Tables

Eine **Rainbow Table** ist eine vorab berechnete Tabelle, die gängige Passwörter mit ihren Hashwerten verknüpft, um bei einem Datenleck sofort nachschlagen zu können, statt jeden Hash einzeln zu berechnen. Ein individueller Salt pro Konto macht vorab berechnete Rainbow Tables praktisch wirkungslos, da für jeden möglichen Salt-Wert eine eigene Tabelle nötig wäre.

Brute Force und Wörterbuchangriffe

Brute Force probiert systematisch jede mögliche Zeichenkombination durch, **Wörterbuchangriffe** probieren stattdessen gezielt bekannte, häufig genutzte Passwörter und deren naheliegende Varianten. Beide Techniken werden in Kapitel 10 anhand von Hashcat und John the Ripper praktisch vertieft.

Passwortentropie und Passwortkomplexität

Entropie misst in Bit, wie unvorhersehbar ein Passwort ist; sie steigt mit der Länge und der Vielfalt der genutzten Zeichen. Ein langes, aus zufälligen Wörtern zusammengesetztes Passwort hat oft eine höhere Entropie als ein kurzes, komplex wirkendes Passwort mit erzwungenen Sonderzeichen, weil Länge entropietechnisch stärker wirkt als erzwungene Zeichenvielfalt.

Argon2id, bcrypt, scrypt und PBKDF2

Verfahren	Besonderheit
Argon2id	Gewinner der Password Hashing Competition 2015, aktuell empfohlener Standard, hoher Speicherbedarf erschwert GPU-Angriffe gezielt
bcrypt	seit 1999 bewährt, konfigurierbarer Kostenfaktor, weiterhin weit verbreitet
scrypt	wie Argon2id speicherintensiv konzipiert, älterer Vorläufer
PBKDF2	älteres, von NIST spezifiziertes Verfahren, rein rechenintensiv statt speicherintensiv, dadurch anfälliger für spezialisierte Hardware

Warum schnelle Hashfunktionen für Passwortspeicherung ungeeignet sind

SHA-256 ist absichtlich schnell, weil Dateiintegrität (Kapitel 2) von schneller Berechnung profitiert. Genau diese Schnelligkeit macht SHA-256 für

Passwort-Hashes ungeeignet: Auf spezialisierter Hardware lassen sich damit, je nach konkretem Modell und Implementierung, in der Größenordnung von Milliarden Kandidaten pro Sekunde durchprobieren. Argon2id, bcrypt und scrypt sind absichtlich langsam und zusätzlich speicherintensiv konfigurierbar, sodass dieselbe Hardware auf dieselben Verfahren angewendet in der Größenordnung nur noch tausende statt Milliarden Versuche pro Sekunde schafft. Diese Zahlen sind illustrative Größenordnungen, keine belastbaren Messwerte; das tatsächliche Verhältnis hängt von Hardware, Softwareversion und den gewählten Kostenparametern ab.

> **WARNUNG – Sicherheitsrisiko**
>
> Verwende niemals SHA-256, SHA-512 oder MD5 allein zur Passwortspeicherung, auch nicht mit Salt. Diese Verfahren sind für Geschwindigkeit optimiert, Passwort-Hashing braucht das genaue Gegenteil.

Moderne Passwortspeicherung und typische Implementierungsfehler

Eine moderne Implementierung speichert pro Konto Salt, verwendetes Verfahren samt Kostenparametern, und den resultierenden Hash gemeinsam in einem Feld, oft im sogenannten PHC-String-Format. Typische Fehler sind ein global statt pro Konto vergebener Salt, ein zu niedrig gewählter Kostenfaktor aus Performancegründen, oder die Verwendung eines schnellen Hashverfahrens aus Unwissenheit.

Praxisübungen

Übung A: Eigene Passwort-Hashes mit bcrypt erzeugen

Lernziel: den Unterschied zwischen einem schnellen und einem absichtlich langsamen Hashverfahren spüren. **Voraussetzungen:** Linux mit installiertem `mkpasswd` (Paket `whois`) oder Python mit `bcrypt`-Modul.

Befehle zu Übung A

```
$ mkpasswd -m bcrypt "MeinTestpasswort123"
```

Erwartetes Ergebnis: Ein Hashwert im Format `$2b$...`, dessen Berechnung spürbar länger dauert als ein `sha256sum`-Aufruf. **Erklärung:** Das Präfix `$2b$` kennzeichnet bcrypt, gefolgt vom Kostenfaktor und dem eingebetteten Salt. **Troubleshooting:** `mkpasswd` nicht gefunden? Mit `sudo apt install whois` nachinstallieren.

Übung B: Salt nachvollziehen

Lernziel: denselben Klartext zweimal hashen und den Unterschied durch Salting beobachten.

Befehle zu Übung B

```
$ mkpasswd -m bcrypt "MeinTestpasswort123"
$ mkpasswd -m bcrypt "MeinTestpasswort123"
```

Erwartetes Ergebnis: Zwei vollständig unterschiedliche Hashwerte für denselben Klartext. **Erklärung:** Jeder Aufruf erzeugt einen neuen, zufälligen Salt, der direkt in den resultierenden Hash eingebettet wird. **Troubleshooting:** Identische Hashwerte bei beiden Aufrufen? Das würde auf eine fehlerhafte oder deaktivierte Zufallszahlenerzeugung hindeuten, ein in der Praxis sehr seltener Fall.

Übung C: Passwort-Hashverfahren vergleichen

Lernziel: die Rechenzeit von SHA-256 und bcrypt gegenüberstellen.

Befehle zu Übung C

```
$ time (echo -n "MeinTestpasswort123" | sha256sum)
$ time mkpasswd -m bcrypt "MeinTestpasswort123"
```

Erwartetes Ergebnis: SHA-256 benötigt Sekundenbruchteile, bcrypt spürbar länger, abhängig vom voreingestellten Kostenfaktor. **Erklärung:** Genau dieser Zeitunterschied, auf Millionen oder Milliarden Versuche hochgerechnet, entscheidet in Kapitel 10 über Erfolg oder Misserfolg eines Cracking-Versuchs. **Troubleshooting:** Kaum messbarer Unterschied? Kostenfaktor von bcrypt mit `-R` erhöhen, um den Effekt deutlicher zu machen.

Zusammenfassung

Zusammenfassung

- Systeme speichern Passwort-Hashes, nicht Passwörter, und vergleichen bei der Anmeldung neu berechnete mit gespeicherten Hashwerten.
- Ein individueller Salt pro Konto macht Rainbow Tables praktisch wirkungslos.
- Argon2id, bcrypt und scrypt sind absichtlich langsam und speicherintensiv, PBKDF2 ist rein rechenintensiv.
- Schnelle Hashfunktionen wie SHA-256 sind für Passwortspeicherung grundsätzlich ungeeignet.

Kontrollfragen

1. Was ist ein Salt? (leicht)
2. Nenne zwei moderne, für Passwörter geeignete Hashverfahren. (leicht)
3. Warum ist SHA-256 für Passwort-Hashes ungeeignet? (mittel)
4. Was unterscheidet einen Pepper von einem Salt? (mittel)
5. Eine Anwendung speichert Passwörter mit SHA-256 und einem für alle Konten identischen Salt. Welche zwei Schwächen liegen hier gleichzeitig vor? (schwer)

Lösungshinweise: 1. Ein zufälliger, pro Konto einzigartiger Wert, der vor dem Hashen an das Passwort angehängt wird. 2. Argon2id und bcrypt (auch scrypt zulässig). 3. Weil es absichtlich schnell ist und einem Angreifer Milliarden Versuche pro Sekunde erlaubt. 4. Ein Pepper ist ein einziges, anwendungsweites, getrennt aufbewahrtes Geheimnis, ein Salt ist pro Konto einzigartig und nicht geheim. 5. Erstens die Verwendung eines für Geschwindigkeit statt Sicherheit optimierten Hashverfahrens, zweitens ein globaler statt individueller Salt, der eine einzige Rainbow Table gegen alle Konten gleichzeitig wirksam macht.

Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] erklären kannst, warum Passwörter gehasht statt verschlüsselt gespeichert werden,

[ ] Salt und Pepper klar voneinander abgrenzen kannst,

[ ] eigene bcrypt-Hashes erzeugt und deren Rechenzeit mit SHA-256 verglichen hast,

[ ] mindestens zwei typische Implementierungsfehler bei der Passwortspeicherung nennen kannst.

Kapitel 4: Symmetrische Verschlüsselung

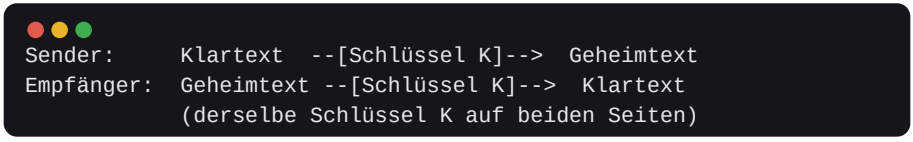
Lernziele

- Du erklärst das Grundprinzip symmetrischer Verschlüsselung an einem Alltagsbeispiel.
- Du unterscheidest die Betriebsmodi ECB, CBC, CTR und GCM.
- Du verschlüsselst und entschlüsselst eine Datei mit AES, mit OpenSSL für ECB/CBC und mit Python für das authentifizierte AES-GCM.

Grundprinzip

Stell dir eine Truhe vor, die mit genau einem Schlüssel auf- und zugeschlossen wird. Wer die Truhe verschließt und wer sie öffnet, braucht denselben Schlüssel. Genau das ist **symmetrische Verschlüsselung**: Derselbe, geheim gehaltene Schlüssel verschlüsselt und entschlüsselt die Daten.

Diagramm 4.1: Symmetrische Verschlüsselung



Gemeinsamer geheimer Schlüssel

Die größte praktische Herausforderung symmetrischer Verfahren ist nicht die Verschlüsselung selbst, sondern die **Schlüsselverteilung**: Wie gelangt derselbe geheime Schlüssel sicher zu beiden Seiten, ohne dass ihn jemand auf dem Weg dorthin mitliest? Kapitel 5 löst genau dieses Problem mit asymmetrischer Kryptographie.

AES

AES (Advanced Encryption Standard) ist der seit 2001 von NIST standardisierte, heute mit Abstand am weitesten verbreitete symmetrische Algorithmus. Er arbeitet mit Schlüssellängen von 128, 192 oder 256 Bit.

Variante	Schlüssellänge	Praxisrelevanz
AES-128	128 Bit	Standard, gilt bis auf absehbare Zeit als sicher
AES-192	192 Bit	selten genutzt, liegt zwischen den beiden anderen Varianten
AES-256	256 Bit	höchste Sicherheitsmarge, oft für besonders sensible Daten gewählt

Blockchiffren und Betriebsmodi

AES ist eine **Blockchiffre**: Sie verschlüsselt Daten in festen Blöcken von 128 Bit, nicht Bit für Bit wie eine Stromchiffre. Da reale Daten fast nie exakt in einen einzigen Block passen, legt ein **Betriebsmodus** fest, wie mehrere Blöcke miteinander verknüpft werden.

Diagramm 4.2: ECB gegenüber CBC (stark vereinfacht)

```
ECB (unsicher für strukturierte Daten):
  Block 1 -> Schlüssel -> Geheimtext 1
  Block 2 -> Schlüssel -> Geheimtext 2
  (identische Klartextblöcke ergeben identische Geheimtextblöcke)

CBC (sicherer):
  Block 1 XOR IV      -> Schlüssel -> Geheimtext 1
  Block 2 XOR Geheimtext 1 -> Schlüssel -> Geheimtext 2
  (jeder Block hängt vom vorherigen ab, identische Klartextblöcke
   ergeben unterschiedliche Geheimtextblöcke)
```

Modus	Grundidee	Einschätzung
ECB (Electronic Codebook)	jeder Block unabhängig verschlüsselt	unsicher für strukturierte Daten, da Muster im Klartext im Geheimtext sichtbar bleiben
CBC (Cipher Block Chaining)	jeder Block wird vor der Verschlüsselung mit dem vorherigen Geheimtextblock verknüpft	sicherer als ECB, aber ohne zusätzliche Maßnahme ohne Integritätsschutz
CTR (Counter Mode)	verwandelt die Blockchiffre in eine Stromchiffre über einen hochzählenden Zähler	parallelisierbar, ebenfalls ohne eingebauten Integritätsschutz
GCM (Galois/Counter Mode)	kombiniert CTR-artige Verschlüsselung mit eingebauter Integritätsprüfung	heutiger Standard für die meisten neuen Anwendungen, da Vertraulichkeit und Integrität gemeinsam abgedeckt werden

> **WARNUNG – Sicherheitsrisiko**
>
> Das berühmte Beispiel für die Schwäche von ECB ist ein verschlüsseltes Bild, bei dem trotz Verschlüsselung die ursprünglichen Umrisse weiterhin erkennbar bleiben, weil gleichartige Bildbereiche zu gleichartigen Geheimtextblöcken führen. Verwende ECB nie für strukturierte oder wiederkehrende Daten.

Initialisierungsvektor, Nonce und Authenticated Encryption

Ein **Initialisierungsvektor** (IV) ist ein einmalig genutzter Startwert, der verhindert, dass dieselbe Klartext-Schlüssel-Kombination immer denselben Geheimtext ergibt; er muss nicht geheim bleiben, aber pro Verschlüsselung

einzigartig sein. Eine **Nonce** erfüllt bei Stromchiffren und CTR-ähnlichen Modi denselben Zweck. **Authenticated Encryption** (wie bei GCM) liefert zusätzlich zum Geheimtext ein Prüfelement, das jede nachträgliche Manipulation erkennbar macht, nicht nur die Vertraulichkeit sichert.

Schlüsselgenerierung und Schlüsselaufbewahrung

Schlüssel müssen mit einem kryptographisch sicheren Zufallszahlengenerator erzeugt werden (vertieft in Kapitel 12), nicht mit vorhersagbaren Werten wie einem aktuellen Zeitstempel. Sie sollten getrennt von den verschlüsselten Daten aufbewahrt werden, idealerweise in einem dedizierten Schlüsseltresor statt in einer Konfigurationsdatei im Klartext.

Typische Sicherheitsfehler

Ein wiederverwendeter Initialisierungsvektor, die Nutzung von ECB für strukturierte Daten, und ein im Quellcode fest hinterlegter Schlüssel gehören zu den häufigsten, in der Praxis real gefundenen Fehlern bei symmetrischer Verschlüsselung.

Praxisübungen

> **WARNUNG – Sicherheitsrisiko**
>
> Das Kommandozeilenwerkzeug `openssl enc` unterstützt keine authentifizierten Modi wie GCM oder CCM; die OpenSSL-Dokumentation weist das ausdrücklich aus und sieht das auch für künftige Versionen nicht vor. Ein `openssl enc -aes-256-gcm`-Aufruf scheitert deshalb unabhängig von der installierten Version. Übung A und B nutzen daher bewusst eine kleine Python-Bibliothek statt `openssl enc`.

Übung A: Eine Datei mit AES-256-GCM verschlüsseln

Lernziel: eine Datei mit einem modernen, authentifizierten Modus tatsächlich lauffähig verschlüsseln. **Voraussetzungen:** Python 3 mit dem Paket `cryptography` (`pip install cryptography`).

Befehle zu Übung A

```
$ echo "Vertrauliche Testdaten" > geheim.txt
$ python3 aesgcm_verschluesseln.py
```

Datei aesgcm_verschluesseln.py

```
from getpass import getpass
from os import urandom
from pathlib import Path
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC
from cryptography.hazmat.primitives import hashes

password = getpass("Passwort: ").encode("utf-8")
salt = urandom(16)
nonce = urandom(12)
kdf = PBKDF2HMAC(algorithm=hashes.SHA256(), length=32, salt=salt,
    iterations=600000)
key = kdf.derive(password)

plaintext = Path("geheim.txt").read_bytes()
ciphertext = AESGCM(key).encrypt(nonce, plaintext, None)
Path("geheim.enc").write_bytes(salt + nonce + ciphertext)
```

Erwartetes Ergebnis: Eine Datei `geheim.enc` mit binärem, nicht lesbarem Inhalt. **Erklärung:** PBKDF2 (Kapitel 3) leitet aus dem eingegebenen Passwort und einem zufälligen Salt den eigentlichen 256-Bit-AES-Schlüssel ab. Die Nonce muss pro Verschlüsselung einzigartig sein (siehe weiter oben in diesem Kapitel) und wird zusammen mit Salt und Geheimtext in derselben Datei gespeichert, da sie zum Entschlüsseln erneut gebraucht wird, aber selbst nicht geheim sein muss. **Troubleshooting:** `ModuleNotFoundError: cryptography`? Mit `pip install cryptography` nachinstallieren, bei Bedarf in einer virtuellen Umgebung (`python3 -m venv venv && source venv/bin/activate`).

Übung B: Die Datei wieder entschlüsseln

Lernziel: den vollständigen Kreislauf aus Ver- und Entschlüsselung nachvollziehen, einschließlich einer erkannten Manipulation.

Datei aesgcm_entschluesseln.py

```
from getpass import getpass
from pathlib import Path
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC
from cryptography.hazmat.primitives import hashes

password = getpass("Passwort: ").encode("utf-8")
data = Path("geheim.enc").read_bytes()
salt, nonce, ciphertext = data[:16], data[16:28], data[28:]
kdf = PBKDF2HMAC(algorithm=hashes.SHA256(), length=32, salt=salt,
    iterations=600000)
key = kdf.derive(password)

plaintext = AESGCM(key).decrypt(nonce, ciphertext, None)
Path("entschluesselt.txt").write_bytes(plaintext)
```

Befehle zu Übung B

```
$ python3 aesgcm_entschluesseln.py
$ diff geheim.txt entschluesselt.txt
```

Erwartetes Ergebnis: `diff` zeigt keine Ausgabe, die entschlüsselte Datei ist identisch mit dem Original. **Erklärung:** Salt und Nonce werden aus den ersten Bytes der Datei gelesen, derselbe Schlüssel wird daraus erneut abgeleitet. Verändere versuchsweise ein einzelnes Byte in `geheim.enc` (zum Beispiel mit einem Hex-Editor) und führe das Skript erneut aus: `AESGCM.decrypt` bricht mit einer `InvalidTag`-Ausnahme ab, statt stillschweigend falsche Daten zu liefern, weil GCM Vertraulichkeit und Integritätsprüfung gemeinsam absichert. **Troubleshooting:** `cryptography.exceptions.InvalidTag`? Entweder ein falsches Passwort oder eine tatsächliche Manipulation der Datei seit der Verschlüsselung, siehe oben. Dieses Beispiel dient der Demonstration des Verfahrens; für produktive Anwendungen sollten Parameter wie die Iterationszahl anhand der aktuellen Bibliotheksdokumentation geprüft werden.

Übung C: ECB und CBC gegenüberstellen

Lernziel: den Unterschied zwischen beiden Modi an sich wiederholenden Daten beobachten.

Befehle zu Übung C

```
$ printf 'AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA' > wiederholt.txt
$ openssl enc -aes-128-ecb -nopad -nosalt -K
    00112233445566778899aabbccddeeff \
    -in wiederholt.txt -out ecb.enc
$ xxd ecb.enc
```

Erwartetes Ergebnis: Die Hex-Ausgabe zeigt zwei identische 16-Byte-Blöcke, da beide Klartextblöcke identisch waren. **Erklärung:** `printf` ohne Leerzeichen erzeugt exakt 32 gleiche Zeichen, also genau zwei identische 16-Byte-Blöcke. `-nopad` verhindert, dass OpenSSL zusätzlich einen dritten, reinen Padding-Block anhängt, der das Ergebnis sonst um einen weiteren, in diesem Fall nicht aussagekräftigen Block verlängern würde. Genau das Muster aus zwei identischen Geheimtextblöcken ist der praktische Beleg für die in Diagramm 4.2 erklärte Schwäche von ECB. **Troubleshooting:** Kein Muster erkennbar? Prüfen, ob die Eingabedatei tatsächlich exakt 32 Zeichen ohne Leer- oder Sonderzeichen enthält und `-nopad` gesetzt ist.

Zusammenfassung

> **Zusammenfassung**
>
> • Symmetrische Verschlüsselung nutzt denselben Schlüssel zum Ver- und Entschlüsseln; die Schlüsselverteilung bleibt die zentrale Herausforderung.
> • AES ist der heutige Standardalgorithmus mit 128, 192 oder 256 Bit Schlüssellänge.
> • ECB ist für strukturierte Daten unsicher; GCM kombiniert Vertraulichkeit mit eingebauter Integritätsprüfung.
> • Initialisierungsvektoren und Nonces müssen pro Verschlüsselung einzigartig sein, auch wenn sie nicht geheim bleiben müssen.

Kontrollfragen

1. Was bedeutet „symmetrisch" bei symmetrischer Verschlüsselung? (leicht)
2. Welche drei Schlüssellängen unterstützt AES? (leicht)

3. Warum gilt ECB als unsicher für strukturierte Daten? (mittel)

4. Was leistet GCM zusätzlich zu reiner Vertraulichkeit? (mittel)

5. Ein Entwickler verschlüsselt mehrere Datenbankeinträge mit CBC und demselben, festen Initialisierungsvektor für jeden Eintrag. Welches konkrete Risiko entsteht dadurch? (schwer)

Lösungshinweise: 1. Derselbe Schlüssel wird zum Ver- und Entschlüsseln verwendet. 2. 128, 192 und 256 Bit. 3. Weil identische Klartextblöcke zu identischen Geheimtextblöcken führen und so Muster im Original sichtbar bleiben. 4. Eine eingebaute Integritätsprüfung, die nachträgliche Manipulation des Geheimtextes erkennbar macht. 5. Einträge mit identischem Beginn würden zu identischen ersten Geheimtextblöcken führen, wodurch ein Angreifer ohne den Schlüssel zu kennen bereits erkennen könnte, welche Einträge mit demselben Inhalt beginnen.

Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] das Grundprinzip symmetrischer Verschlüsselung erklären kannst,
- [ ] ECB, CBC, CTR und GCM anhand ihrer jeweiligen Eigenschaften unterscheiden kannst,
- [ ] eine Datei mit Python und AES-256-GCM verschlüsselt und wieder entschlüsselt hast,
- [ ] die Schwäche von ECB an einem eigenen Beispiel praktisch gezeigt hast.

Kapitel 5: Asymmetrische Verschlüsselung

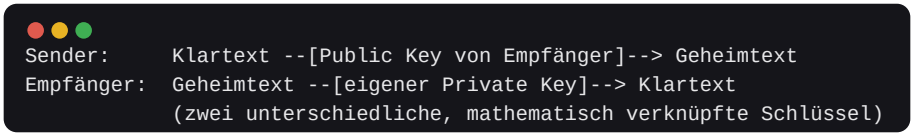
Lernziele

- Du erklärst das Grundprinzip von Public und Private Key an einem Alltagsbeispiel.
- Du ordnest RSA, Diffie-Hellman und ECC ihrem jeweiligen Haupteinsatzzweck zu.
- Du erzeugst ein eigenes RSA-Schlüsselpaar und nutzt es zur Verschlüsselung.

Grundprinzip

Stell dir einen Briefkasten mit einem Schlitz vor, durch den jeder einen Brief einwerfen kann, den aber nur der Besitzer mit seinem Schlüssel wieder öffnen kann. Genau das ist **asymmetrische Verschlüsselung**: Ein öffentlich bekannter Schlüssel verschlüsselt, nur der dazu passende, geheim gehaltene Schlüssel entschlüsselt.

Diagramm 5.1: Asymmetrische Verschlüsselung



Public Key, Private Key und Schlüsselpaar

Ein **Schlüsselpaar** besteht aus einem **Public Key** (öffentlich, darf jeder kennen) und einem **Private Key** (geheim, verlässt nie den Besitzer). Beide sind mathematisch so verknüpft, dass mit dem Public Key verschlüsselte Daten ausschließlich mit dem passenden Private Key entschlüsselt werden können. Dieses Prinzip löst das Schlüsselverteilungsproblem aus Kapitel 4: Der Public Key kann offen übertragen werden, ohne die Vertraulichkeit zu gefährden.

RSA

RSA, benannt nach seinen Erfindern Rivest, Shamir und Adleman, beruht auf der mathematischen Schwierigkeit, eine sehr große Zahl, das Produkt zweier großer Primzahlen, wieder in diese beiden Primfaktoren zu zerlegen. Mit heutiger Rechenleistung ist diese Faktorisierung für ausreichend große Zahlen (2048 Bit oder mehr) praktisch nicht in sinnvoller Zeit durchführbar.

Diffie-Hellman

Der **Diffie-Hellman-Schlüsselaustausch** löst ein anderes Problem als RSA: Er erlaubt zwei Parteien, über einen unsicheren Kanal einen gemeinsamen, geheimen Wert zu vereinbaren, ohne diesen Wert selbst jemals zu übertragen. Dieser gemeinsame Wert dient anschließend oft als symmetrischer Sitzungsschlüssel (Kapitel 8).

Diagramm 5.2: Diffie-Hellman-Grundprinzip (stark vereinfacht)

```
Alice wählt geheime Zahl a  ->  sendet öffentlichen Wert A
Bob wählt geheime Zahl b    ->  sendet öffentlichen Wert B
Alice berechnet aus B und a denselben gemeinsamen Wert wie
Bob aus A und b -- ohne dass a oder b je übertragen wurden
```

Elliptic Curve Cryptography, ECC und ECDH

ECC (Elliptic Curve Cryptography) ist der Oberbegriff für Verfahren, die auf dem diskreten Logarithmusproblem elliptischer Kurven beruhen, statt wie RSA auf Faktorisierung, und dieselbe Sicherheit mit deutlich kürzeren Schlüsseln erreichen. **ECDH** (beziehungsweise seine verbreitete Ausprägung X25519) ist die ECC-Variante des Diffie-Hellman-Schlüsselaustauschs aus diesem Kapitel.

> **Gut zu wissen**
>
> ECC ist ein Oberbegriff, kein einzelnes Verfahren mit einem einzigen Einsatzzweck. Für Schlüsselaustausch steht ECDH beziehungsweise X25519, für digitale Signaturen stehen ECDSA und Ed25519 – Letzteres wird in Kapitel 6 behandelt. Ed25519 selbst ist ein Signaturverfahren, kein Werkzeug zum Verschlüsseln oder zum Schlüsselaustausch; es taucht deshalb hier bewusst nicht als ECC-Beispiel für Verschlüsselung auf.

Verfahren	Zweck	Typische Schlüssellänge für etwa 128 Bit klassische Sicherheit
RSA	Verschlüsselung, RSA-Schlüsseltransport (historisch), Signaturen	etwa 3072 Bit
ECDH / X25519	Schlüsselaustausch	256 Bit
ECDSA / Ed25519	digitale Signaturen (Kapitel 6)	256 Bit

> **Gut zu wissen**
>
> Diese Werte sind grobe, gerundete Anhaltspunkte für ein vergleichbares Sicherheitsniveau, keine exakten Äquivalenzen; sie orientieren sich an gängigen Einschätzungen von NIST und BSI. Für eine konkrete, aktuelle Empfehlung lohnt sich ein Blick in die jeweils neueste Veröffentlichung dieser Stellen.

Schlüsselverteilung und Schlüsselgenerierung

Da der Public Key offen übertragen werden darf, entfällt das in Kapitel 4 beschriebene Verteilungsproblem für den Schlüssel selbst. Es bleibt aber die Frage, ob ein empfangener Public Key tatsächlich der Person gehört, die er vorgibt zu sein – genau dieses Problem löst die PKI in Kapitel 7.

Hybridverschlüsselung

Diagramm 5.3: Hybridverschlüsselung, zwei unterschiedliche Wege

```
Weg A, RSA-Schlüsseltransport (historisch, auch in diesem Kapitel geübt):
1. Ein zufälliger, symmetrischer Sitzungsschlüssel wird erzeugt
2. Die eigentlichen Daten werden mit diesem Schlüssel per AES verschlüsselt
3. Der Sitzungsschlüssel selbst wird mit dem RSA-Public-Key des
   Empfängers verschlüsselt (nur für wenige Byte nötig)
4. Empfänger entschlüsselt zuerst den Sitzungsschlüssel, dann die Daten

Weg B, ECDH/ECDHE (so arbeitet TLS 1.3, Kapitel 8):
1. Beide Seiten besitzen oder erzeugen je ein ECDH-Schlüsselpaar
   und tauschen nur die beiden öffentlichen Werte aus
2. Jede Seite berechnet daraus lokal denselben gemeinsamen,
   geheimen Wert (Diagramm 5.2), ohne ihn zu übertragen
3. Aus diesem gemeinsamen Wert wird der symmetrische Sitzungsschlüssel
   abgeleitet, nicht durch Verschlüsseln, sondern durch Schlüsselableitung
4. Die eigentlichen Daten werden mit diesem Schlüssel per AES verschlüsselt
```

Asymmetrische Verfahren sind für große Datenmengen zu langsam. **Hybridverschlüsselung** kombiniert deshalb beide Welten, auf zwei unterschiedlichen Wegen: Beim klassischen RSA-Schlüsseltransport wird ein kurzer Sitzungsschlüssel tatsächlich mit dem Public Key des Empfängers verschlüsselt. Bei ECDH oder seiner für TLS gebräuchlichen Variante ECDHE wird dagegen gar nichts verschlüsselt; beide Seiten leiten stattdessen über den Diffie-Hellman-Mechanismus aus Diagramm 5.2 denselben gemeinsamen, geheimen Wert ab und gewinnen daraus den Sitzungsschlüssel. TLS 1.3 nutzt ausschließlich den zweiten Weg und verzichtet vollständig auf RSA-Schlüsseltransport, wie Kapitel 8 im Detail zeigt.

Vorteile und Grenzen asymmetrischer Verfahren

Der Vorteil liegt im gelösten Schlüsselverteilungsproblem; die Grenze liegt in der Geschwindigkeit, weshalb Hybridverschlüsselung in der Praxis die Regel, reine asymmetrische Verschlüsselung großer Datenmengen die Ausnahme ist.

Praxisübungen

Übung A: RSA-Schlüsselpaar mit OpenSSL erstellen

Lernziel: ein eigenes, 4096 Bit starkes RSA-Schlüsselpaar erzeugen.

Befehle zu Übung A

```
$ openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 \
    -out private.pem
$ openssl rsa -pubout -in private.pem -out public.pem
```

Erwartetes Ergebnis: Zwei Dateien, `private.pem` und `public.pem`, beide im PEM-Textformat lesbar. **Erklärung:** `genpkey` erzeugt den vollständigen Schlüssel, `rsa -pubout` extrahiert daraus den öffentlichen Teil. **Troubleshooting:** Die Erzeugung dauert ungewöhnlich lange? Das ist bei 4096 Bit normal; für Testzwecke genügt notfalls auch 2048 Bit.

Übung B: Öffentlichen und privaten Schlüssel untersuchen

Lernziel: den Aufbau einer PEM-Datei nachvollziehen.

Befehle zu Übung B

```
$ cat public.pem
$ openssl rsa -in private.pem -text -noout | head -5
```

Erwartetes Ergebnis: `public.pem` zeigt Base64-kodierten Inhalt zwischen `BEGIN PUBLIC KEY` und `END PUBLIC KEY`; der zweite Befehl zeigt die Schlüssellänge im Klartext. **Erklärung:** PEM ist nur eine Base64-Kodierung (Kapitel 1) der eigentlichen, binären Schlüsseldaten, keine Verschlüsselung des Schlüssels selbst. **Troubleshooting:** Private Key ohne Passwortschutz erzeugt? Für produktive Nutzung zusätzlich `-aes256` bei der Erzeugung ergänzen, um den privaten Schlüssel selbst zusätzlich zu schützen.

Übung C: Daten mit dem öffentlichen Schlüssel verschlüsseln

Lernziel: den vollständigen Kreislauf aus Verschlüsselung und Entschlüsselung mit einem Schlüsselpaar durchlaufen.

Befehle zu Übung C

```
$ echo "Geheime Testnachricht" > nachricht.txt
$ openssl pkeyutl -encrypt -pubin -inkey public.pem \
    -in nachricht.txt -out nachricht.enc \
    -pkeyopt rsa_padding_mode:oaep \
    -pkeyopt rsa_oaep_md:sha256 -pkeyopt rsa_mgf1_md:sha256
$ openssl pkeyutl -decrypt -inkey private.pem \
    -in nachricht.enc -out nachricht.dec \
```

```
    -pkeyopt rsa_padding_mode:oaep \
    -pkeyopt rsa_oaep_md:sha256 -pkeyopt rsa_mgf1_md:sha256
$ diff nachricht.txt nachricht.dec
```

Erwartetes Ergebnis: `diff` liefert keine Ausgabe, die entschlüsselte Nachricht entspricht dem Original. **Erklärung:** Nur der zu `public.pem` passende `private.pem` kann die mit dem öffentlichen Schlüssel verschlüsselte Nachricht entschlüsseln. Die drei `-pkeyopt`-Optionen setzen ausdrücklich das moderne OAEP-Padding mit SHA-256 statt eines unausgesprochenen Standardwerts; Verschlüsselung und Entschlüsselung müssen dabei dieselben Padding-Optionen verwenden. **Troubleshooting:** Fehlermeldung zur Datengröße? RSA kann direkt nur kleine Datenmengen verschlüsseln, bei 4096 Bit und OAEP mit SHA-256 etwas unter 446 Byte; für größere Dateien ist Hybridverschlüsselung (Diagramm 5.3) der richtige Weg. Fehlermeldung „padding check failed" beim Entschlüsseln? Prüfen, ob bei Ver- und Entschlüsselung exakt dieselben Padding-Optionen gesetzt wurden.

Zusammenfassung

> **Zusammenfassung**
>
> • Asymmetrische Verschlüsselung nutzt ein mathematisch verknüpftes Schlüsselpaar aus Public und Private Key.
> • RSA beruht auf der Schwierigkeit der Primfaktorzerlegung, ECC auf dem diskreten Logarithmusproblem elliptischer Kurven.
> • Diffie-Hellman und ECDH erlauben einen sicheren Schlüsselaustausch über einen unsicheren Kanal.
> • Hybridverschlüsselung kombiniert die Stärken asymmetrischer und symmetrischer Verfahren.

Kontrollfragen

1. Was unterscheidet Public Key von Private Key? (leicht)
2. Worauf beruht die Sicherheit von RSA? (leicht)
3. Warum wird in der Praxis meist Hybridverschlüsselung statt reiner asymmetrischer Verschlüsselung genutzt? (mittel)

4. Was löst der Diffie-Hellman-Schlüsselaustausch konkret? (mittel)

5. Ein öffentlicher Schlüssel allein garantiert noch nicht, dass er tatsächlich zur behaupteten Person gehört. Welches, in einem späteren Kapitel behandelte System löst genau dieses Problem? (schwer)

Lösungshinweise: 1. Der Public Key darf jeder kennen und wird zum Verschlüsseln genutzt, der Private Key bleibt geheim und entschlüsselt. 2. Auf der Schwierigkeit, das Produkt zweier großer Primzahlen wieder zu faktorisieren. 3. Weil asymmetrische Verfahren für große Datenmengen zu langsam sind. 4. Es erlaubt zwei Parteien, über einen unsicheren Kanal einen gemeinsamen, geheimen Wert zu vereinbaren, ohne ihn zu übertragen. 5. Die Public Key Infrastructure aus Kapitel 7, die über Zertifikate die Zuordnung eines Public Keys zu einer tatsächlichen Identität bestätigt.

Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

[ ] das Grundprinzip von Public und Private Key erklären kannst,

[ ] RSA, Diffie-Hellman und ECC ihrem jeweiligen Haupteinsatzzweck zuordnest,

[ ] ein eigenes RSA-Schlüsselpaar erzeugt und zur Verschlüsselung genutzt hast,

[ ] erklären kannst, warum Hybridverschlüsselung in der Praxis die Regel ist.

Kapitel 6: Digitale Signaturen

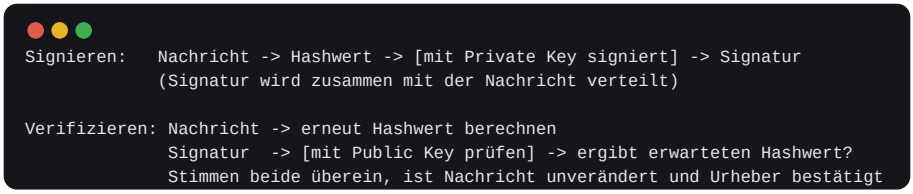
Lernziele

- Du erklärst, wie eine digitale Signatur Integrität und Authentizität gleichzeitig sichert.
- Du unterscheidest digitale Signaturen klar von Verschlüsselung.
- Du signierst eine Datei, veränderst sie, und weist die Manipulation nach.

Was ist eine digitale Signatur?

Eine **digitale Signatur** ist das kryptographische Gegenstück zu einer handschriftlichen Unterschrift, aber deutlich stärker: Sie bestätigt sowohl, wer eine Nachricht erstellt hat, als auch, dass sie seitdem nicht verändert wurde.

Diagramm 6.1: Signieren und Verifizieren



Hashwert und Signatur

Statt die gesamte, möglicherweise sehr große Nachricht zu signieren, wird zunächst ihr Hashwert (Kapitel 2) berechnet und nur dieser kurze, feste Wert signiert. Das ist deutlich schneller und ändert nichts an der Aussagekraft, da schon eine einzige veränderte Stelle in der Nachricht den Hashwert vollständig verändert.

Signieren und Verifizieren

Signieren geschieht mit dem **Private Key** des Urhebers, Verifizieren mit dem passenden **Public Key**. Das ist die exakt umgekehrte Schlüsselzuordnung gegenüber der Verschlüsselung aus Kapitel 5, bei der der Public Key verschlüsselt und der Private Key entschlüsselt.

> **Gut zu wissen**
>
> Dieser Unterschied verwirrt häufig: Bei Verschlüsselung verschlüsselt der Public Key, bei Signaturen signiert der Private Key. Das ist eine nützliche Merkhilfe, aber keine vollständige technische Beschreibung: Signieren ist nicht einfach Verschlüsseln in umgekehrter Richtung. Signaturverfahren wie ECDSA oder Ed25519 sind eigenständige mathematische Konstruktionen mit eigenen Berechnungsschritten, nicht einfach ein mit dem privaten Schlüssel verschlüsselter Hashwert. Nur bei RSA lässt sich die klassische Lehrbuchvariante der Signatur tatsächlich als Verschlüsselung des Hashwerts mit dem privaten Schlüssel auffassen; das ist eine Besonderheit von RSA, keine allgemeine Eigenschaft digitaler Signaturen.

Integritätsprüfung und Authentizität

Eine gültige Signatur bestätigt zwei der vier Sicherheitsziele aus Kapitel 1 gleichzeitig: **Integrität**, weil jede Veränderung der Nachricht den Hashwert und damit die Gültigkeit der Signatur zerstört, und **Authentizität**, weil nur der Besitzer des passenden Private Keys eine gültige Signatur erzeugen kann.

Unterschied zwischen Verschlüsselung und Signatur

Verschlüsselung sichert **Vertraulichkeit**: Niemand ohne den passenden Schlüssel kann den Inhalt lesen. Eine Signatur sichert **Integrität und Authentizität**, verbirgt den Inhalt aber nicht; eine signierte, aber unverschlüsselte Nachricht bleibt für jeden lesbar. Beide Mechanismen lassen sich kombinieren, erfüllen aber unterschiedliche Zwecke.

RSA-Signaturen, ECDSA und Ed25519

RSA lässt sich, mit angepasstem Verfahren, sowohl zur Verschlüsselung als auch zur Signatur nutzen. **ECDSA** ist das auf elliptischen Kurven basierende Signaturverfahren, oft in TLS-Zertifikaten (Kapitel 7) anzutreffen. **Ed25519** gilt als moderne, besonders robuste Wahl, unter anderem weil es widerstandsfähiger gegen bestimmte Implementierungsfehler bei der Erzeugung von Zufallswerten ist als klassisches ECDSA.

Typische Einsatzgebiete

Digitale Signaturen bestätigen die Echtheit digitaler Dokumente (etwa bei elektronisch unterzeichneten Verträgen), sichern Software-Updates gegen nachträgliche Manipulation ab, und bilden die Grundlage jedes TLS-Zertifikats in Kapitel 7, da eine Zertifizierungsstelle ein Zertifikat genau mit ihrer eigenen digitalen Signatur bestätigt.

Technische Prüfung und rechtliche Verbindlichkeit

Eine erfolgreiche Verifizierung, wie in Übung B gezeigt, beweist ausschließlich, dass die Daten zu einem bestimmten Schlüsselpaar passen und seit der Signierung unverändert geblieben sind. Sie beweist für sich allein nicht, welche natürliche Person den privaten Schlüssel tatsächlich bedient hat, ob dieser Schlüssel jemals gestohlen oder missbräuchlich verwendet wurde, oder welche rechtliche Wirkung die Signatur in einem bestimmten Land oder Kontext hat. Rechtliche Nichtabstreitbarkeit setzt zusätzlich eine verlässliche Zuordnung des Schlüssels zu einer Identität voraus (vergleiche die Zertifikate aus Kapitel 7) sowie einen nachweislich geschützten Umgang mit dem privaten Schlüssel. Dieser Abschnitt ist eine allgemeine, technische Einordnung und keine Rechtsberatung zur konkreten rechtlichen Wirkung elektronischer Signaturen in einem bestimmten Land.

Praxisübungen

Übung A: Eine Datei signieren

Lernziel: eine eigene Datei mit dem in Kapitel 5 erzeugten Schlüsselpaar signieren. **Voraussetzungen:** das Schlüsselpaar `private.pem`/`public.pem` aus Kapitel 5, Übung A.

Befehle zu Übung A

```
$ echo "Dieser Vertrag ist gültig." > vertrag.txt
$ openssl dgst -sha256 -sign private.pem -out vertrag.sig
    vertrag.txt
```

Erwartetes Ergebnis: Eine binäre Signaturdatei `vertrag.sig`. **Erklärung:** `dgst -sha256` berechnet zunächst den SHA-256-Hashwert der Datei, `-sign` signiert anschließend genau diesen Hashwert mit dem privaten

Schlüssel. **Troubleshooting:** Fehlermeldung zum Schlüsselformat? Prüfen, ob `private.pem` tatsächlich im PEM-Format vorliegt und nicht beschädigt wurde.

Übung B: Die Signatur überprüfen

Lernziel: eine gültige Signatur erfolgreich verifizieren.

Befehle zu Übung B

```
$ openssl dgst -sha256 -verify public.pem -signature vertrag.sig
    vertrag.txt
```

Erwartetes Ergebnis: Die Ausgabe `Verified OK`. **Erklärung:** OpenSSL berechnet den Hashwert der aktuellen Datei neu und prüft, ob die Signatur zu genau diesem Hashwert und dem öffentlichen Schlüssel passt. **Troubleshooting:** `Verification Failure`? Prüfen, ob tatsächlich der zu `private.pem` passende `public.pem` verwendet wurde.

Übung C: Manipulation erkennen

Lernziel: nachweisen, dass eine nachträgliche Änderung die Signatur ungültig macht.

Befehle zu Übung C

```
$ echo "Zusätzlicher, nicht vereinbarter Satz." >> vertrag.txt
$ openssl dgst -sha256 -verify public.pem -signature vertrag.sig
    vertrag.txt
```

Erwartetes Ergebnis: Die Ausgabe `Verification Failure`, obwohl dieselbe Signaturdatei wie in Übung B verwendet wurde. **Erklärung:** Der Avalanche-Effekt (Kapitel 2) sorgt dafür, dass bereits eine angehängte Zeile den Hashwert und damit die Übereinstimmung mit der ursprünglichen Signatur zerstört. **Troubleshooting:** Verifizierung gelingt trotz Änderung? Prüfen, ob tatsächlich dieselbe, nun veränderte Datei `vertrag.txt` genutzt wurde, nicht versehentlich eine unveränderte Kopie.

Zusammenfassung

Zusammenfassung

• Eine digitale Signatur entsteht aus dem signierten Hashwert einer Nachricht, nicht aus der vollständigen Nachricht selbst.
• Signieren nutzt den Private Key, Verifizieren den Public Key, umgekehrt zur Verschlüsselung.
• Eine Signatur sichert Integrität und Authentizität, nicht Vertraulichkeit.
• Jede nachträgliche Änderung der signierten Daten macht die Signatur ungültig.

Kontrollfragen

1. Welcher Schlüssel signiert, welcher verifiziert? (leicht)
2. Welche zwei Sicherheitsziele erfüllt eine gültige Signatur? (leicht)
3. Warum wird nicht die gesamte Nachricht, sondern ihr Hashwert signiert? (mittel)
4. Warum sichert eine Signatur nicht automatisch Vertraulichkeit? (mittel)
5. Ein signiertes Software-Update wird von einem Angreifer abgefangen und durch eine manipulierte Version ersetzt, ohne die ursprüngliche Signatur zu verändern. Würde die Verifizierung beim Nutzer trotzdem fehlschlagen, und warum? (schwer)

Lösungshinweise: 1. Der Private Key signiert, der Public Key verifiziert. 2. Integrität und Authentizität. 3. Weil das schneller ist und der Avalanche-Effekt dieselbe Aussagekraft wie eine Signatur der vollständigen Nachricht liefert. 4. Weil eine Signatur den Inhalt nicht verbirgt, sondern nur seine Herkunft und Unverändertheit bestätigt. 5. Ja, die Verifizierung würde fehlschlagen, weil die unveränderte, alte Signatur nicht zum Hashwert der manipulierten, neuen Version passt; ein Angreifer müsste zusätzlich den privaten Schlüssel des Herausgebers besitzen, um eine gültige neue Signatur zu erzeugen.

Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

[ ] erklären kannst, wie eine Signatur Integrität und Authentizität gleichzeitig sichert,

[ ] Signatur und Verschlüsselung klar voneinander abgrenzen kannst,

[ ] eine eigene Datei signiert und die Signatur erfolgreich verifiziert hast,

[ ] eine Manipulation nach der Signierung anhand einer fehlgeschlagenen Verifizierung nachgewiesen hast.

Kapitel 7: Public Key Infrastructure und Zertifikate

Lernziele

- Du erklärst, welches Problem eine PKI löst, das reine asymmetrische Kryptographie offenlässt.
- Du verstehst den Aufbau einer Zertifikatskette von Root-CA bis Endzertifikat.
- Du richtest eine eigene Test-CA ein und stellst damit ein Testzertifikat aus.

Was ist eine PKI?

Kapitel 5 hat gezeigt: Ein Public Key allein garantiert nicht, dass er tatsächlich der behaupteten Person gehört. Eine **Public Key Infrastructure** (PKI) löst genau dieses Vertrauensproblem, indem eine dritte, als vertrauenswürdig anerkannte Partei die Zuordnung zwischen Public Key und Identität bestätigt.

Diagramm 7.1: Grundidee einer PKI

```
Ohne PKI:  "Hier ist mein Public Key" -> woher weißt du, dass das stimmt?

Mit PKI:   "Hier ist mein Public Key, bestätigt durch das Zertifikat
            einer vertrauenswürdigen Stelle, der dein System bereits
            vertraut" -> Vertrauen wird auf die Zertifizierungsstelle
            übertragen, nicht auf eine unbestätigte Behauptung
```

Certificate Authority, Root CA und Intermediate CA

Eine **Certificate Authority** (CA, Zertifizierungsstelle) stellt Zertifikate aus und signiert sie mit ihrem eigenen Private Key. Eine **Root CA** steht an der Spitze dieser Vertrauenskette; ihr Zertifikat ist in Betriebssystemen und Browsern bereits vorinstalliert und als vertrauenswürdig hinterlegt. Eine **Intermediate CA** wird von der Root CA selbst zertifiziert und stellt ihrerseits Endzertifikate aus, damit der besonders sensible Private Key der Root CA im Alltagsbetrieb nicht ständig genutzt werden muss.

X.509-Zertifikate

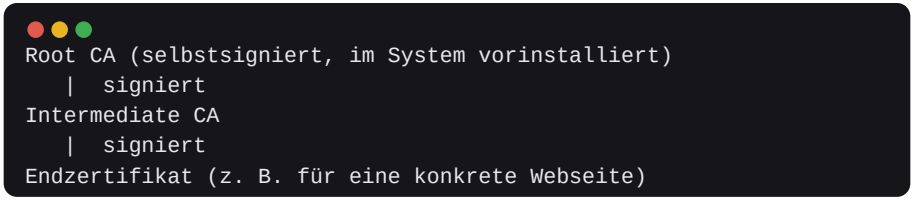
X.509 (spezifiziert in RFC 5280) ist der heute praktisch überall genutzte Standard für digitale Zertifikate. Ein X.509-Zertifikat enthält unter anderem den Public Key des Inhabers, dessen Identität, die ausstellende CA, einen Gültigkeitszeitraum, und die Signatur der ausstellenden CA über all diese Angaben.

Certificate Signing Request

Ein **CSR** (Certificate Signing Request) ist die formale Anfrage, die ein Antragsteller an eine CA sendet: Er enthält den eigenen, bereits generierten Public Key und Identitätsangaben, aber noch keine Signatur der CA. Der zugehörige Private Key verlässt beim gesamten Vorgang nie den Antragsteller.

Zertifikatsausstellung und Zertifikatskette

Diagramm 7.2: Eine typische Zertifikatskette



Ein Browser prüft bei einer HTTPS-Verbindung die gesamte **Zertifikatskette** bis zu einer ihm bereits bekannten Root CA zurück. Fehlt ein Glied dieser Kette oder lässt sich die Signatur nicht bis zu einer vertrauenswürdigen Root CA zurückverfolgen, meldet der Browser einen Zertifikatsfehler.

Subject und Subject Alternative Name

Das **Subject** eines Zertifikats benennt formal den Inhaber. Der **Subject Alternative Name** (SAN) listet zusätzlich alle Domainnamen auf, für die das Zertifikat tatsächlich gültig ist; moderne Browser prüfen praktisch ausschließlich den SAN, nicht mehr das ältere Subject-Feld allein.

Gültigkeitszeitraum, Zertifikatswiderruf, CRL und OCSP

Jedes Zertifikat hat einen begrenzten **Gültigkeitszeitraum**. Wird ein Zertifikat vor Ablauf kompromittiert, etwa weil der private Schlüssel gestohlen wurde, muss es widerrufen werden können. Eine **CRL** (Certificate Revocation List) ist eine von der CA veröffentlichte Liste widerrufener Zertifikate. **OCSP** (Online Certificate Status Protocol) erlaubt stattdessen, den Status eines einzelnen Zertifikats gezielt und aktuell bei der CA abzufragen, ohne eine vollständige Liste herunterladen zu müssen.

Schlüsselverwaltung und Hardware Security Module

Der private Schlüssel einer CA ist das wertvollste Gut der gesamten Vertrauenskette: Wer ihn besitzt, kann beliebige, scheinbar gültige Zertifikate ausstellen. Ein **HSM** (Hardware Security Module) ist spezialisierte Hardware, die private Schlüssel so verwahrt, dass sie das Gerät physisch nie im Klartext verlassen, selbst Operationen wie Signieren laufen innerhalb des HSM ab.

Typische Zertifikatsfehler

Fehler	Beschreibung
Abgelaufenes Zertifikat	Gültigkeitszeitraum überschritten
Domainname stimmt nicht überein	SAN enthält nicht die tatsächlich aufgerufene Domain
Unvollständige Kette	Intermediate-Zertifikat fehlt auf dem Server
Selbstsigniertes Zertifikat ohne Importierung	Root CA dem aufrufenden System nicht bekannt

Praxisübungen

Übung A: Eigene Test-CA erstellen

Lernziel: eine selbstsignierte Root CA für das eigene Labor aufbauen, mit ausdrücklich gesetzten CA-Erweiterungen statt impliziter Standardwerte aus der lokalen OpenSSL-Konfiguration.

Befehle zu Übung A

```
$ openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096
    -out ca.key
$ openssl req -new -x509 -key ca.key -days 3650 -out ca.crt \
    -subj "/CN=Meine Test-CA/O=Testlabor" \
    -addext "basicConstraints=critical,CA:TRUE" \
    -addext "keyUsage=critical,keyCertSign,cRLSign" \
    -addext "subjectKeyIdentifier=hash"
```

Erwartetes Ergebnis: Ein privater Schlüssel `ca.key` und ein selbstsigniertes Root-Zertifikat `ca.crt` mit den drei CA-Erweiterungen; prüfbar mit `openssl x509 -in ca.crt -text -noout`. **Erklärung:** `-x509` erzeugt direkt ein selbstsigniertes Zertifikat statt eines CSR, passend für eine Root CA, die sich selbst bestätigt. `basicConstraints=CA:TRUE` kennzeichnet das Zertifikat ausdrücklich als CA-fähig, `keyUsage` beschränkt den Schlüssel auf das Signieren von Zertifikaten und Sperrlisten, `subjectKeyIdentifier` erleichtert späteren Systemen den Aufbau der Zertifikatskette. Viele OpenSSL-Installationen setzen `CA:TRUE` über ihre mitgelieferte Konfigurationsdatei ohnehin automatisch; die ausdrücklichen `-addext`-Angaben machen den Befehl aber unabhängig von einer bestimmten, lokal vorgefundenen Konfiguration. **Troubleshooting:** Zu lange Gültigkeit für ein Testlabor? `-days` kann für Übungszwecke problemlos niedriger gesetzt werden. Fehlermeldung zu `-addext`? Diese Option benötigt mindestens OpenSSL 1.1.1; prüfe die Version mit `openssl version`.

Übung B: Zertifikatsanfrage generieren und Testzertifikat ausstellen

> **WARNUNG – Sicherheitsrisiko**
>
> Ein Zertifikat, das ohne Subject Alternative Name und ohne passende Erweiterungen ausgestellt wird, ist zwar gültig signiert, aber für eine normale HTTPS-Serverprüfung ungeeignet, da moderne Clients den SAN prüfen (siehe weiter oben in diesem Kapitel). Diese Übung ergänzt deshalb eine Extensions-Datei.

Lernziel: den vollständigen CSR-Workflow nachvollziehen und ein tatsächlich HTTPS-taugliches Zertifikat ausstellen.

Datei server_ext.cnf

```
basicConstraints = critical, CA:FALSE
keyUsage = critical, digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth
subjectAltName = DNS:testserver.local
```

Befehle zu Übung B

```
$ openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048
    -out server.key
$ openssl req -new -key server.key -out server.csr \
    -subj "/CN=testserver.local"
$ openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key \
    -CAcreateserial -days 365 -out server.crt \
    -extfile server_ext.cnf
```

Erwartetes Ergebnis: Ein Endzertifikat `server.crt`, signiert von der eigenen Test-CA, mit SAN und den für einen TLS-Server passenden Erweiterungen. **Erklärung:** Der Server erzeugt seinen eigenen Schlüssel und CSR; die CA signiert nur den öffentlichen Teil, der private Schlüssel `server.key` verlässt nie den Server. `-extfile` sorgt dafür, dass die in `server_ext.cnf` festgelegten Erweiterungen tatsächlich ins ausgestellte Zertifikat übernommen werden; ohne diese Angabe übernimmt `x509 -req` standardmäßig keine Erweiterungen aus dem CSR. **Troubleshooting:** Fehlermeldung zur Seriennummer? `-CAcreateserial` muss beim allerersten ausgestellten Zertifikat einer CA gesetzt sein. SAN fehlt trotzdem im Ergebnis? Mit `openssl x509 -in server.crt -text -noout | grep -A1 "Subject Alternative Name"` prüfen, ob `-extfile` tatsächlich beim Ausstellen angegeben wurde.

Übung C: Zertifikatskette untersuchen und Hostnamen prüfen

Lernziel: die Angaben im ausgestellten Zertifikat lesen und verstehen, und Kettenprüfung von Hostnamenprüfung unterscheiden.

Befehle zu Übung C

```
$ openssl x509 -in server.crt -text -noout | head -20
$ openssl verify -CAfile ca.crt server.crt
$ openssl verify -CAfile ca.crt -verify_hostname testserver.local
    server.crt
```

Erwartetes Ergebnis: Beide `verify`-Aufrufe melden `server.crt: OK`. **Erklärung:** Der zweite Aufruf allein prüft nur, ob sich die Signaturkette des Zertifikats bis zur angegebenen CA zurückverfolgen lässt; das sagt noch nichts darüber aus, ob das Zertifikat auch zum tatsächlich verwendeten Namen passt. Erst `-verify_hostname` prüft zusätzlich, ob `testserver.local` tatsächlich im Subject Alternative Name steht, genau der SAN aus Übung B. **Troubleshooting:** Meldung „unable to get local issuer certificate"? Prüfen, ob `-CAfile` tatsächlich auf die korrekte, ausstellende CA zeigt. Meldung zum Hostnamen trotz gültiger Kette? Prüfen, ob der SAN in `server_ext.cnf` exakt dem bei `-verify_hostname` angegebenen Namen entspricht.

Zusammenfassung

> **Zusammenfassung**
>
> • Eine PKI überträgt Vertrauen in einen Public Key auf eine bereits als vertrauenswürdig anerkannte Zertifizierungsstelle.
> • X.509-Zertifikate verknüpfen Public Key, Identität und Gültigkeitszeitraum, bestätigt durch die Signatur einer CA.
> • Eine Zertifikatskette muss sich lückenlos bis zu einer vertrauenswürdigen Root CA zurückverfolgen lassen.
> • CRL und OCSP erlauben, widerrufene Zertifikate vor Ablauf ihrer eigentlichen Gültigkeit zu erkennen.

Kontrollfragen

1. Was ist eine Root CA? (leicht)
2. Was enthält ein CSR, und was nicht? (leicht)
3. Warum nutzen viele PKIs eine Intermediate CA statt direkt mit der Root CA zu signieren? (mittel)
4. Was unterscheidet CRL von OCSP? (mittel)
5. Ein Server präsentiert ein gültig signiertes Endzertifikat, aber ohne das zugehörige Intermediate-Zertifikat. Warum kann die Verbindung trotzdem fehlschlagen, obwohl das Endzertifikat selbst korrekt ist? (schwer)

Lösungshinweise: 1. Die an der Spitze einer Vertrauenskette stehende, im System bereits als vertrauenswürdig hinterlegte Zertifizierungsstelle. 2. Den öffentlichen Schlüssel und Identitätsangaben des Antragstellers, aber keine Signatur der CA. 3. Um den besonders sensiblen privaten Schlüssel der Root CA im Alltagsbetrieb zu schonen und seltener einsetzen zu müssen. 4. CRL ist eine vollständige, heruntergeladene Liste widerrufener Zertifikate, OCSP fragt den Status eines einzelnen Zertifikats gezielt und aktuell ab. 5. Ohne das Intermediate-Zertifikat kann der Client die Signaturkette nicht lückenlos bis zur ihm bekannten Root CA zurückverfolgen, selbst wenn das Endzertifikat für sich genommen korrekt signiert wurde.

Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] erklären kannst, welches Vertrauensproblem eine PKI löst,
- [ ] den Aufbau einer Zertifikatskette von Root-CA bis Endzertifikat skizzieren kannst,
- [ ] eine eigene Test-CA eingerichtet und damit ein Testzertifikat ausgestellt hast,
- [ ] CRL und OCSP in ihrem jeweiligen Zweck unterscheiden kannst.

Kapitel 8: TLS und HTTPS verständlich erklärt

Lernziele

- Du erklärst den Ablauf eines TLS-1.3-Handshakes in eigenen Worten.
- Du unterscheidest TLS 1.2 und TLS 1.3 anhand ihrer wichtigsten Unterschiede.
- Du untersuchst eine echte TLS-Verbindung mit OpenSSL und liest das Zertifikat einer Webseite aus.

Warum TLS benötigt wird

Ohne TLS würde jede HTTP-Verbindung im Klartext laufen, lesbar für jeden, der den Datenverkehr mitschneidet (vergleiche das Wireshark-Handbuch dieser Fachbibliothek). TLS verschlüsselt die Verbindung selbst, unabhängig vom übertragenen Inhalt.

Unterschied zwischen SSL und TLS

SSL (Secure Sockets Layer) war der historische Vorläufer, dessen letzte Version SSL 3.0 seit Jahren als unsicher gilt und von keinem modernen System mehr akzeptiert werden sollte. **TLS** (Transport Layer Security) ist der Nachfolger und seit Version 1.0 der tatsächlich verwendete Standard; der Name „SSL-Zertifikat" hat sich umgangssprachlich gehalten, obwohl technisch längst TLS gemeint ist.

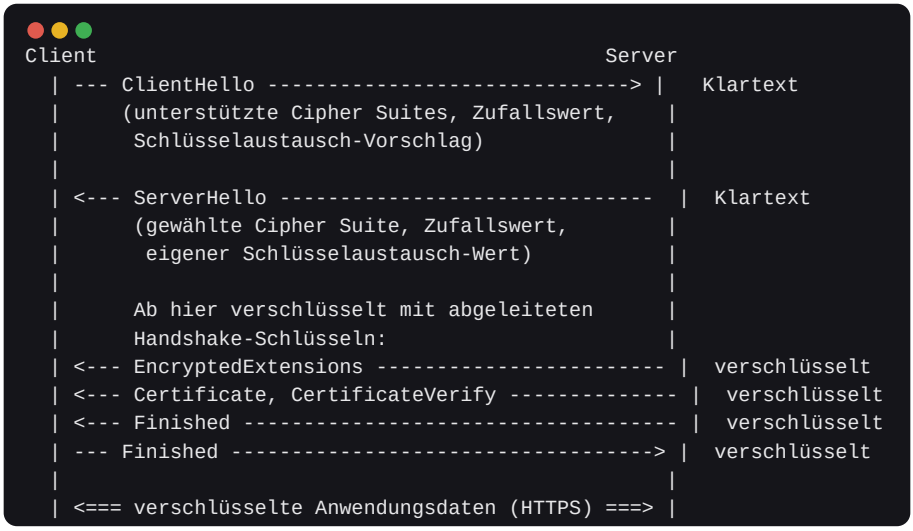
Entwicklung der TLS-Versionen

Version	Status
TLS 1.0, TLS 1.1	veraltet, von modernen Browsern nicht mehr unterstützt
TLS 1.2	weiterhin weit verbreitet und sicher bei korrekter Konfiguration

Version	Status
TLS 1.3	aktueller Standard (RFC 8446), schnellerer Handshake, veraltete Optionen entfernt

Der TLS-1.3-Handshake

Diagramm 8.1: TLS-1.3-Handshake (vereinfacht)



ClientHello eröffnet die Verbindung und schlägt unterstützte Verfahren samt einem Schlüsselaustausch-Wert vor. **ServerHello** antwortet mit der gewählten Cipher Suite und dem eigenen Schlüsselaustausch-Wert; beide sichtbaren Nachrichten laufen noch im Klartext. Bereits aus diesen beiden Werten leiten Client und Server denselben Satz an Handshake-Schlüsseln ab (Kapitel 5, Diffie-Hellman-Prinzip), sodass alle weiteren Handschlag-Nachrichten, einschließlich des Serverzertifikats, bereits verschlüsselt übertragen werden. Das Zertifikat steht deshalb nicht im ServerHello selbst, sondern in einer eigenen, bereits verschlüsselten Certificate-Nachricht kurz danach. TLS 1.2 benötigte für denselben Ablauf typischerweise eine zusätzliche, unverschlüsselte Runde.

Schlüsselaustausch und Sitzungsschlüssel

Der eigentliche Inhalt der Verbindung wird, wie in Kapitel 5 als Hybridverschlüsselung beschrieben, nicht asymmetrisch, sondern mit einem während des Handshakes vereinbarten, symmetrischen **Sitzungsschlüssel** verschlüsselt, typischerweise mit AES-GCM (Kapitel 4).

Zertifikatsprüfung

Während des Handshakes prüft der Client das vom Server präsentierte Zertifikat vollständig: Gültigkeitszeitraum, passender Domainname im Subject Alternative Name, und eine lückenlose Zertifikatskette bis zu einer bekannten Root CA (Kapitel 7).

Forward Secrecy

Forward Secrecy bedeutet, dass für jede einzelne Sitzung ein neuer, unabhängiger Sitzungsschlüssel ausgehandelt wird. Selbst wenn der private Schlüssel des Servers später gestohlen würde, ließen sich damit keine in der Vergangenheit aufgezeichneten, bereits abgeschlossenen Sitzungen nachträglich entschlüsseln.

> **Gut zu wissen**
>
> Diese Eigenschaft gilt für den in Diagramm 8.1 gezeigten, frischen (EC)DHE-Schlüsselaustausch, den Standardfall einer neuen TLS-1.3-Verbindung. Zwei Ausnahmen benennt RFC 8446 ausdrücklich: Ein reiner PSK-Modus ohne zusätzlichen (EC)DHE-Anteil (`psk_ke`) bietet keine Forward Secrecy, da er sich allein auf einen bereits bekannten, vorab vereinbarten Schlüssel stützt. Auch 0-RTT-Daten, die ein Client bereits mit der allerersten Nachricht mitschickt, um Zeit zu sparen, sind schwächer abgesichert und nicht forward secret. TLS 1.3 erzwingt Forward Secrecy also für den üblichen (EC)DHE-Fall, nicht ausnahmslos für jeden unterstützten Modus.

HTTPS und Cipher Suites

HTTPS ist schlicht HTTP, geführt über eine TLS-verschlüsselte Verbindung. Eine **Cipher Suite** legt fest, welche konkreten kryptographischen Verfahren eine Verbindung tatsächlich verwendet, allerdings mit einem wichtigen Unterschied zwischen den TLS-Versionen: Bei TLS 1.2 bündelte eine Cipher

Suite traditionell Schlüsselaustausch-, Verschlüsselungs- und Signaturverfahren in einem einzigen, zusammenhängenden Bezeichner. TLS 1.3 trennt das auf: Eine TLS-1.3-Cipher-Suite legt nur noch AEAD-Verschlüsselungsalgorithmus und Hashfunktion fest (zum Beispiel `TLS_AES_128_GCM_SHA256`); die Schlüsselaustauschgruppe und das Signaturverfahren werden über eigene, getrennte Erweiterungen im ClientHello ausgehandelt.

Typische TLS-Fehler und unsichere Konfigurationen

Die Unterstützung veralteter Protokollversionen wie TLS 1.0, schwache oder veraltete Cipher Suites, und abgelaufene Zertifikate gehören zu den häufigsten, in der Praxis tatsächlich gefundenen TLS-Fehlkonfigurationen.

> **WARNUNG – Sicherheitsrisiko**
>
> Ein Downgrade-Angriff versucht, eine Verbindung gezielt auf eine ältere, schwächere TLS-Version zu drängen. Moderne Server sollten TLS 1.0 und 1.1 vollständig deaktivieren, nicht nur als unbevorzugt markieren.

Praxisübungen

Übung A: TLS-Verbindung mit OpenSSL untersuchen

Lernziel: eine echte TLS-Verbindung von der Kommandozeile aus prüfen, mit tatsächlich wirksamer Zertifikats- und Hostnamenprüfung.

Befehle zu Übung A

```
$ openssl s_client -connect www.example.com:443 -tls1_3 \
    -verify_return_error -verify_hostname www.example.com
```

Erwartetes Ergebnis: Eine Ausgabe mit der vollständigen Zertifikatskette und der ausgehandelten Protokollversion. **Erklärung:** `-tls1_3` erzwingt diese Version gezielt, um den modernen Handshake aus Diagramm 8.1 nachzuvollziehen. Ohne `-verify_return_error` meldet `s_client` einen Verifikationsfehler zwar in seiner Ausgabe, baut die Verbindung aber trotzdem auf; erst `-verify_return_error` lässt die Verbindung bei einem

tatsächlichen Problem auch abbrechen, und `-verify_hostname` prüft zusätzlich den Hostnamen, nicht nur die Signaturkette (vergleiche Kapitel 7). **Troubleshooting:** Verbindung schlägt fehl? Server unterstützt möglicherweise kein TLS 1.3; Option entfernen und erneut versuchen, um die tatsächlich unterstützte Version zu sehen.

Übung B: Zertifikatsinformationen einer Webseite auslesen

Lernziel: Subject Alternative Name und Gültigkeitszeitraum eines echten Zertifikats lesen.

Befehle zu Übung B

```
$ echo | openssl s_client -connect www.example.com:443 2>/dev/null
    |
    openssl x509 -noout -dates -subject -ext subjectAltName
```

Erwartetes Ergebnis: Gültigkeitszeitraum, Subject und die Liste der im SAN eingetragenen Domainnamen. **Erklärung:** `echo |` beendet die interaktive Verbindung sofort wieder, nachdem das Zertifikat empfangen wurde, sodass sich das Ergebnis direkt weiterverarbeiten lässt. **Troubleshooting:** Keine Ausgabe bei `-ext subjectAltName`? Manche älteren Zertifikate besitzen keinen SAN; das ist selbst ein meldenswerter Befund (vergleiche Kapitel 12).

Übung C: TLS-Handshake mit Wireshark nachvollziehen

Lernziel: die einzelnen Schritte aus Diagramm 8.1 tatsächlich im Netzwerkverkehr sehen. **Voraussetzungen:** Wireshark, ausführlich im gleichnamigen Handbuch dieser Fachbibliothek behandelt.

Vorgehen zu Übung C

```
1. Mitschnitt auf der eigenen Netzwerkschnittstelle starten
2. Display-Filter "tls.handshake" setzen
3. Eine HTTPS-Seite im Browser aufrufen
4. ClientHello und ServerHello im gefilterten Mitschnitt
    identifizieren
```

Erwartetes Ergebnis: Zwei sichtbare Pakete mit den Typen „Client Hello" und „Server Hello", jeweils mit lesbaren, unverschlüsselten Detailfeldern. Alle danach folgenden Handshake-Pakete zeigt Wireshark dagegen nur noch als verschlüsselten Inhalt vom Typ „Application Data" (TLS 1.3 verschleiert ab

hier sogar den eigentlichen Nachrichtentyp), nicht mehr als lesbares Certificate oder Finished. **Erklärung:** Nur ClientHello und ServerHello laufen bei TLS 1.3 im Klartext; direkt danach beginnt, wie in Diagramm 8.1 gezeigt, bereits der verschlüsselte Teil des Handshakes selbst, nicht erst die Anwendungskommunikation. **Troubleshooting:** Keine TLS-Pakete sichtbar? Prüfen, ob der Mitschnitt tatsächlich an der richtigen Netzwerkschnittstelle erfolgt, vergleiche das Wireshark-Handbuch, Kapitel 2. Siehst du nach ServerHello noch ein lesbares Certificate-Paket? Das deutet auf eine über TLS 1.2 ausgehandelte Verbindung hin, nicht auf TLS 1.3.

Zusammenfassung

> **Zusammenfassung**
>
> • TLS verschlüsselt die Verbindung selbst; HTTPS ist HTTP über eine TLS-gesicherte Verbindung.
> • TLS 1.3 benötigt für den Handshake nur eine Hin- und Rückrunde und bietet Forward Secrecy beim üblichen, frischen (EC)DHE-Schlüsselaustausch; PSK-only und 0-RTT sind ausdrücklich genannte Ausnahmen.
> • Während des Handshakes wird das Serverzertifikat vollständig geprüft, bevor Nutzdaten verschlüsselt übertragen werden.
> • Veraltete Protokollversionen und schwache Cipher Suites gehören zu den häufigsten TLS-Fehlkonfigurationen.

Kontrollfragen

1. Was ist der Unterschied zwischen SSL und TLS? (leicht)
2. Was bedeutet HTTPS? (leicht)
3. Was leistet Forward Secrecy? (mittel)
4. Warum braucht TLS 1.3 für den Handshake weniger Runden als TLS 1.2? (mittel)
5. Ein Server unterstützt sowohl TLS 1.3 als auch, aus Kompatibilitätsgründen, TLS 1.0. Welches konkrete Risiko entsteht dadurch, selbst wenn kein Client TLS 1.0 absichtlich auswählt? (schwer)

Lösungshinweise: 1. SSL ist der veraltete, unsichere Vorläufer, TLS der aktuelle, tatsächlich genutzte Standard. 2. HTTP, geführt über eine TLS-verschlüsselte Verbindung. 3. Dass jede Sitzung einen eigenen Sitzungsschlüssel erhält, sodass ein später gestohlener Serverschlüssel keine vergangenen Sitzungen entschlüsseln kann. 4. Weil der Schlüsselaustausch bereits im ersten ClientHello vorgeschlagen und im ServerHello direkt bestätigt werden kann, ohne eine zusätzliche Bestätigungsrunde. 5. Ein Downgrade-Angriff könnte die Verbindung aktiv auf das schwächere TLS 1.0 drängen, selbst wenn beide Seiten eigentlich TLS 1.3 bevorzugen würden; die bloße Verfügbarkeit der alten Version eröffnet dieses Risiko.

Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] den TLS-1.3-Handshake in eigenen Worten erklären kannst,
- [ ] TLS 1.2 und TLS 1.3 anhand ihrer wichtigsten Unterschiede unterscheidest,
- [ ] eine echte TLS-Verbindung mit OpenSSL untersucht hast,
- [ ] ClientHello und ServerHello in einem Wireshark-Mitschnitt identifiziert hast.

Kapitel 9: Kryptographie in der Netzwerkkommunikation

Lernziele

- Du ordnest gängige Verschlüsselungsprotokolle den passenden OSI-Schichten zu.
- Du unterscheidest Transportverschlüsselung von Ende-zu-Ende-Verschlüsselung.
- Du erzeugst ein SSH-Schlüsselpaar und vergleichst verschlüsselten mit unverschlüsseltem Verkehr.

Kryptographie und OSI-Modell

Verschlüsselung findet nicht auf einer einzigen Netzwerkschicht statt, sondern je nach Verfahren auf unterschiedlichen Ebenen des OSI-Modells.

Diagramm 9.1: Kryptographische Verfahren im OSI-Modell (Auswahl)

```
Schicht 7 (Anwendung)     E-Mail-Verschlüsselung, DNS over HTTPS
Schicht 6/5 (Darstellung/Sitzung) TLS/HTTPS, SSH
Schicht 3 (Vermittlung)   IPsec
Schicht 2 (Sicherung)     WPA2/WPA3 (WLAN-Verschlüsselung)
```

Verschlüsselung auf verschiedenen Netzwerkebenen

Je niedriger die Schicht, desto transparenter die Verschlüsselung für darüberliegende Anwendungen, aber desto eingeschränkter ihre Reichweite: WLAN-Verschlüsselung (Schicht 2) schützt nur die Funkstrecke zwischen Gerät und Access Point, nicht die weitere Strecke durchs Internet.

HTTPS, SSH, VPN und IPsec

HTTPS sichert Webverkehr (Kapitel 8). **SSH** (Secure Shell) verschlüsselt Fernzugriffe auf Systeme und nutzt denselben asymmetrischen

Grundmechanismus wie TLS, mit eigenem Schlüsselformat. Ein **VPN** (Virtual Private Network) verschlüsselt den gesamten Datenverkehr zwischen zwei Netzwerken oder einem Gerät und einem Netzwerk. **IPsec** ist ein verbreitetes Protokoll zur Absicherung von VPN-Verbindungen auf der Vermittlungsschicht.

WPA2 und WPA3

WPA2 und sein Nachfolger **WPA3** verschlüsseln WLAN-Verkehr zwischen Endgerät und Access Point. KRACK war eine 2017 veröffentlichte Schwachstelle in Implementierungen des WPA2-Vier-Wege-Handshakes, die einen erzwungenen erneuten Schlüsseleinsatz ausnutzte; sie betraf die Umsetzung des Protokolls in konkreter Software, nicht jede WPA2-Verbindung automatisch und gleichermaßen. WPA3-Personal ersetzt den betroffenen Handshake durch das SAE-Verfahren (Simultaneous Authentication of Equals), das diese spezifische Angriffsklasse schließt und für diesen Modus Forward Secrecy mit sich bringt; die konkreten Eigenschaften hängen aber vom gewählten WPA3-Modus und der jeweiligen Implementierung ab.

DNS over HTTPS und DNS over TLS

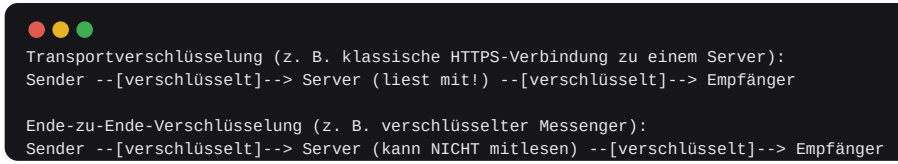
Klassisches DNS überträgt Anfragen unverschlüsselt und ist damit sowohl mitlesbar als auch manipulierbar. **DNS over HTTPS** (DoH) und **DNS over TLS** (DoT) verschlüsseln DNS-Anfragen, mit demselben TLS-Mechanismus aus Kapitel 8, lösen damit allerdings nicht automatisch jedes DNS-Sicherheitsproblem, etwa Fragen der Vertrauenswürdigkeit des gewählten DNS-Anbieters.

E-Mail-Verschlüsselung

E-Mail-Verschlüsselung wie PGP oder S/MIME verschlüsselt den Nachrichteninhalt selbst mit dem Public Key des Empfängers (Hybridverschlüsselung, Kapitel 5), unabhängig davon, über wie viele Mailserver die Nachricht auf dem Weg dorthin läuft.

Ende-zu-Ende-Verschlüsselung gegenüber Transportverschlüsselung

Diagramm 9.2: Transportverschlüsselung gegenüber Ende-zu-Ende



Transportverschlüsselung schützt die Übertragungsstrecke, der zwischengeschaltete Server selbst kann den Inhalt aber einsehen, da er ihn entschlüsselt und neu verschlüsselt weiterleitet.
Ende-zu-Ende-Verschlüsselung verschlüsselt bereits beim Absender mit dem Schlüssel des tatsächlichen Empfängers, sodass kein zwischengeschalteter Dienst den Klartext jemals einsehen kann.

Verschlüsselung im Vergleich

Verfahren	Schützt primär
HTTPS/TLS	Verbindung zu einem bestimmten Server
VPN/IPsec	gesamten Datenverkehr zwischen zwei Netzwerkpunkten
WPA2/WPA3	nur die WLAN-Funkstrecke
Ende-zu-Ende (Messenger, PGP)	Inhalt selbst, unabhängig von der Übertragungsstrecke

Typische Netzwerkfehlkonfigurationen

Ein offenes WLAN ohne jede Verschlüsselung, ein VPN, das nur einen Teil des Datenverkehrs tatsächlich durch den Tunnel leitet (Split Tunneling ohne Bewusstsein für die Konsequenzen), und eine Anwendung, die fälschlich Transportverschlüsselung mit Ende-zu-Ende-Verschlüsselung gleichsetzt, gehören zu den verbreitetsten Fehleinschätzungen in der Praxis.

Praxisübungen

Übung A: SSH-Schlüssel erstellen

Lernziel: ein modernes SSH-Schlüsselpaar erzeugen.

Befehle zu Übung A

```
$ ssh-keygen -t ed25519 -C "testlabor@kryptographie"
```

Erwartetes Ergebnis: Ein privater (`id_ed25519`) und ein öffentlicher (`id_ed25519.pub`) Schlüssel im Verzeichnis `~/.ssh`. **Erklärung:** Ed25519 (Kapitel 5) gilt gegenüber dem älteren RSA als modernere, ebenso sichere, aber schnellere Wahl für SSH-Schlüssel. **Troubleshooting:** Bereits ein Schlüssel mit diesem Namen vorhanden? Mit `-f` einen alternativen Dateinamen angeben, um bestehende Schlüssel nicht zu überschreiben.

Übung B: HTTPS-Verbindungen analysieren

Lernziel: die in Kapitel 8 geübte Zertifikatsprüfung auf eine selbst gewählte Seite anwenden.

Befehle zu Übung B

```
$ echo | openssl s_client -connect www.example.com:443 2>/dev/null
    |
    openssl x509 -noout -issuer -dates
```

Erwartetes Ergebnis: Ausstellende CA und Gültigkeitszeitraum des echten Zertifikats dieser Seite. **Erklärung:** Dieselbe Technik wie in Kapitel 8, hier als Wiederholung im Netzwerkkontext dieses Kapitels. **Troubleshooting:** siehe Kapitel 8, Übung B.

Übung C: Verschlüsselte und unverschlüsselte Verbindungen vergleichen

Lernziel: mit Wireshark den sichtbaren Unterschied zwischen HTTP und HTTPS beobachten. **Voraussetzungen:** Wireshark, vergleiche das gleichnamige Handbuch dieser Fachbibliothek.

Vorgehen zu Übung C

```
1. Mitschnitt starten, eine reine HTTP-Testseite aufrufen
   (z. B. eine eigene, lokale Testumgebung ohne TLS)
2. Display-Filter "http" setzen, Anfrage im Klartext lesen
3. Denselben Mitschnitt für eine HTTPS-Seite wiederholen,
```

```
Display-Filter "tls" setzen
```

Erwartetes Ergebnis: Bei HTTP ist die vollständige Anfrage lesbar, bei HTTPS nur verschlüsselte Anwendungsdaten. **Erklärung:** Dieser direkte Vergleich macht den praktischen Wert von TLS sichtbar, der in Kapitel 8 nur beschrieben wurde. **Troubleshooting:** Keine reine HTTP-Testseite vorhanden? Ein einfacher, lokaler Webserver ohne TLS-Konfiguration genügt für diesen Vergleich.

Zusammenfassung

> **Zusammenfassung**
>
> • Verschlüsselung findet auf unterschiedlichen OSI-Schichten statt, von WLAN-Verschlüsselung bis Anwendungsebene.
> • Transportverschlüsselung schützt die Übertragungsstrecke, Ende-zu-Ende-Verschlüsselung den Inhalt selbst.
> • VPN und IPsec verschlüsseln den gesamten Datenverkehr zwischen zwei Netzwerkpunkten, nicht nur eine einzelne Verbindung.
> • WPA3 schließt bekannte Schwächen von WPA2 und erzwingt zusätzlich Forward Secrecy für WLAN-Verbindungen.

Kontrollfragen

1. Was unterscheidet Transportverschlüsselung von Ende-zu-Ende-Verschlüsselung? (leicht)
2. Auf welcher OSI-Schicht wirkt WPA2/WPA3? (leicht)
3. Warum reicht WLAN-Verschlüsselung allein nicht für die gesamte Übertragungsstrecke? (mittel)
4. Was leisten DNS over HTTPS und DNS over TLS? (mittel)
5. Ein Unternehmen nutzt einen Cloud-Messenger mit Transportverschlüsselung und bewirbt ihn fälschlich als „Ende-zu-Ende-verschlüsselt". Welches konkrete Risiko übersieht ein Nutzer, der dieser Aussage vertraut? (schwer)

Lösungshinweise: 1. Transportverschlüsselung lässt einen zwischengeschalteten Server den Inhalt einsehen,

Ende-zu-Ende-Verschlüsselung nicht. 2. Schicht 2 (Sicherungsschicht). 3. Weil sie nur die Funkstrecke zwischen Gerät und Access Point schützt, nicht die weitere Strecke durchs Internet. 4. Sie verschlüsseln DNS-Anfragen und verhindern damit Mitlesen und Manipulation auf dem Übertragungsweg. 5. Der Nutzer könnte fälschlich annehmen, der Anbieter selbst könne den Inhalt nicht einsehen, obwohl bei reiner Transportverschlüsselung genau der Anbieter als zwischengeschalteter Server technisch dazu in der Lage wäre.

Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] gängige Protokolle den passenden OSI-Schichten zuordnen kannst,
- [ ] Transportverschlüsselung klar von Ende-zu-Ende-Verschlüsselung abgrenzen kannst,
- [ ] ein eigenes SSH-Schlüsselpaar erzeugt hast,
- [ ] den sichtbaren Unterschied zwischen HTTP- und HTTPS-Verkehr in Wireshark beobachtet hast.

Kapitel 10: Kryptographie und Ethical Hacking – Hashcat und John the Ripper

Lernziele

- Du erklärst, warum Hashcat und John the Ripper erst durch die Kryptographie aus Kapitel 2 und 3 verständlich werden.
- Du identifizierst ein unbekanntes Hashformat anhand seiner Länge und seines Präfixes.
- Du misst praktisch, wie stark Salt und Kostenfaktor die Cracking-Geschwindigkeit beeinflussen.

Einordnung statt Wiederholung

Das Hashcat-Handbuch und das John-the-Ripper-Handbuch dieser Fachbibliothek erklären ausführlich, wie beide Werkzeuge bedient werden. Dieses Kapitel wiederholt das nicht, sondern vertieft die kryptographische Grundlage, die ihre Funktionsweise überhaupt erst erklärt: Warum ist ein Hashwert ein lohnendes Angriffsziel, und warum fallen manche Hashverfahren leichter als andere?

Was ist Hashcat, was ist John the Ripper?

Hashcat ist stark auf beschleunigte, parallele Kandidatenprüfung ausgelegt und nutzt dafür typischerweise die Rechenleistung von Grafikkarten (GPUs). **John the Ripper** bietet traditionell besonders breite Formatunterstützung und flexible, scriptbare Workflows. Beide Werkzeuge können, abhängig vom konkreten Build, dem Hashformat und der verfügbaren Hardware, auch andere Beschleuniger nutzen; die GPU- beziehungsweise CPU-Zuordnung beschreibt eine typische Schwerpunktsetzung, keine strikte, ausschließliche Eigenschaft. Beide verfolgen dasselbe kryptographische Grundprinzip: einen Kandidaten hashen und mit einem Ziel-Hashwert vergleichen, in sehr hoher Wiederholungszahl.

Hashidentifikation und unterstützte Hashformate

Bevor ein Hash überhaupt geprüft werden kann, muss das verwendete Verfahren identifiziert werden. Präfix und Länge liefern dafür erste Anhaltspunkte, aber keine eindeutige Identifikation: Mehrere unterschiedliche Verfahren erzeugen zum Teil gleich lange Hexadezimal-Ausgaben, eine reine Hex-Zeichenkette ohne Präfix lässt sich deshalb oft nicht sicher nur einem einzigen Verfahren zuordnen.

Beobachtung	Mögliche Kandidaten
32 Hexadezimalzeichen	unter anderem MD5 (Kapitel 2)
40 Hexadezimalzeichen	unter anderem SHA-1 (Kapitel 2)
64 Hexadezimalzeichen	unter anderem SHA-256 (Kapitel 2)
Beginnt mit `$2b$`, `$2a$` oder `$2y$`	bcrypt (Kapitel 3)
Beginnt mit `$argon2id$`	Argon2id (Kapitel 3)
Beginnt mit `$6$`	SHA-512-Crypt (Linux-Systempasswörter)

> **Gut zu wissen**
>
> Ein Präfix mit festem Formatmarker, wie bei bcrypt, Argon2id oder dem modularen Crypt-Format allgemein, identifiziert das Verfahren deutlich zuverlässiger als eine reine Zeichenlänge. Eine Hex-Zeichenkette ohne erkennbaren Präfix bleibt dagegen ein Indiz, kein Beweis; Werkzeuge wie `hashid` schlagen deshalb bei langen Hex-Werten oft mehrere mögliche Kandidaten gleichzeitig vor.

Passwort-Hashes untersuchen und offline Passwortprüfung

Ein Angreifer, der eine Datenbank mit Passwort-Hashes erbeutet, muss nicht online, Versuch für Versuch, gegen das eigentliche System anmelden. Die **Offline-Passwortprüfung** berechnet stattdessen lokal, ohne jede Rückmeldung des Zielsystems, Kandidaten durch und vergleicht sie mit dem

gestohlenen Hash – genau deshalb ist die in Kapitel 3 behandelte Rechenkosten-Erhöhung durch Argon2id oder bcrypt die wirksamste Gegenmaßnahme.

Wörterbuchangriffe, regelbasierte Angriffe und Maskenangriffe

Diagramm 10.1: Drei Angriffsstrategien im Vergleich (Konzept)

```
Wörterbuch:   "passwort", "sommer2024", "qwertz123", ...
              (bekannte, häufig genutzte Wörter direkt durchprobieren)

Regelbasiert: "passwort" -> "Passwort", "passwort!", "p4ssw0rt", ...
              (ein Wörterbuch wird nach festen Mustern automatisch variiert)

Maske:        ?u?l?l?l?l?l?d?d?d?d
              (ein Großbuchstabe, sechs Kleinbuchstaben, vier Ziffern,
               probiert jede Kombination genau in diesem Muster durch)
```

Brute Force im eigentlichen Sinn probiert den gesamten Zeichenraum ohne jede Einschränkung durch, was bei längeren Passwörtern schnell unpraktikabel wird. Eine **Maske** grenzt diesen Raum gezielt auf ein plausibles Muster ein, etwa wenn ein Unternehmen bekanntermaßen Passwörter nach einem festen Schema vergibt.

GPU und CPU

Eine GPU besitzt tausende einfache Rechenkerne, die sich hervorragend für massenhaft parallele, gleichartige Berechnungen eignen, wie sie etwa bei MD5 oder SHA-256 anfallen. Genau das macht Hashcat gegen schnelle Hashfunktionen besonders wirkungsvoll, und genau das ist der Grund, warum Argon2id absichtlich hohen Speicherbedarf statt reiner Rechenlast nutzt: Hoher Speicherbedarf pro einzelner Berechnung lässt sich auf einer GPU nicht beliebig parallelisieren, da der verfügbare Speicher begrenzt bleibt.

Warum manche Hashverfahren leichter angreifbar sind

Die in Kapitel 3 eingeführten Kostenfaktoren bestimmen direkt, wie viele Kandidaten pro Sekunde überhaupt geprüft werden können. Ein

ungesalzener MD5-Hash lässt sich zudem gegen eine vorab berechnete Rainbow Table (Kapitel 3) nachschlagen, ein gesalzener Argon2id-Hash dagegen nicht.

Einfluss von Salt, Iterationen und Rechenkosten

Faktor	Wirkung auf einen Cracking-Versuch
Salt vorhanden	macht vorab berechnete Tabellen wirkungslos, erzwingt Berechnung pro Konto einzeln
hoher Kostenfaktor (Iterationen)	verlangsamt jeden einzelnen Versuch proportional
hoher Speicherbedarf (Argon2id)	begrenzt parallele GPU-Berechnung zusätzlich zur reinen Rechenzeit

Argon2id und moderne Passwortspeicherung

Ein mit Argon2id und angemessenen Parametern gehashtes, zufälliges Passwort ausreichender Länge (Kapitel 3) bleibt auch mit spezialisierter Hashcat-Hardware in praktisch relevanter Zeit ungeknackt. Die Verteidigung aus Kapitel 3 und der Angriff aus diesem Kapitel sind damit zwei Seiten derselben kryptographischen Medaille.

Grenzen des Passwort-Crackings

Passwort-Cracking findet nur Passwörter, die tatsächlich im durchprobierten Raum liegen, sei es ein Wörterbuch, eine Regelvariante oder eine Maske. Ein ausreichend langes, tatsächlich zufälliges Passwort kann durch keine dieser Strategien in praktisch relevanter Zeit gefunden werden, unabhängig von der eingesetzten Hardware.

Schutzmaßnahmen gegen Passwortangriffe

Lange, zufällige Passwörter, moderne Hashverfahren mit individuellem Salt, und eine zusätzliche Kontosperrung nach mehreren Fehlversuchen für Online-Angriffe (vertieft in den Windows- und Linux-Security-Handbüchern

dieser Fachbibliothek) ergänzen sich zu einem wirksamen Gesamtschutz.

Praxisübungen

> **WARNUNG – Sicherheitsrisiko**
>
> Führe alle folgenden Übungen ausschließlich mit selbst erzeugten Testdaten und auf eigenen Testsystemen durch.

Übung A: Eigenen Test-Hash erzeugen und identifizieren

Lernziel: einen Hash anhand von Länge und Präfix korrekt einordnen.

Befehle zu Übung A

```
$ echo -n "testpasswort" | sha1sum
$ echo -n "testpasswort" | sha256sum
```

Erwartetes Ergebnis: Zwei unterschiedlich lange Hexadezimalwerte, 40 und 64 Zeichen. **Erklärung:** Genau diese Längenangabe ist der erste Schritt jeder Hashidentifikation, wie in der Tabelle dieses Kapitels gezeigt. **Troubleshooting:** siehe Kapitel 2, Übung D.

Übung B: Testpasswort mit Hashcat untersuchen

Lernziel: einen Wörterbuchangriff gegen einen selbst erzeugten Hash durchführen. **Voraussetzungen:** Hashcat installiert, ausführlich im gleichnamigen Handbuch dieser Fachbibliothek behandelt.

Befehle zu Übung B

```
$ echo -n "passwort123" | sha256sum | cut -d' ' -f1 > testhash.txt
$ hashcat -a 0 -m 1400 testhash.txt
    /usr/share/wordlists/rockyou.txt
```

Erwartetes Ergebnis: Hashcat findet das Testpasswort, da es in gängigen Wörterbüchern enthalten ist. **Erklärung:** `-m 1400` wählt den Hashcat-Modus für reinen SHA-256; die Modusnummer muss exakt zum identifizierten Hashformat passen. **Troubleshooting:** Kein Treffer? Prüfen, ob die Wortliste tatsächlich vorhanden ist; vertieftes Vorgehen im Hashcat-Handbuch dieser Fachbibliothek.

Übung C: John the Ripper im eigenen Labor einsetzen

Lernziel: denselben Hash mit dem alternativen Werkzeug prüfen.

Befehle zu Übung C

```
$ john --format=raw-sha256 testhash.txt
```

Erwartetes Ergebnis: Dasselbe gefundene Passwort wie in Übung B, mit einem anderen Werkzeug. **Erklärung:** Beide Werkzeuge verfolgen dasselbe kryptographische Prinzip; vertieftes Vorgehen im John-the-Ripper-Handbuch dieser Fachbibliothek. **Troubleshooting:** Format nicht erkannt? Mit `john --list=formats` die exakte Bezeichnung für das identifizierte Hashformat nachschlagen.

Übung D: Unterschiedliche Hashverfahren vergleichen

Lernziel: denselben Angriff gegen einen schnellen und einen langsamen Hash gegenüberstellen.

Befehle zu Übung D

```
$ mkpasswd -m bcrypt "passwort123" > testhash_bcrypt.txt
$ time hashcat -a 0 -m 1400 testhash.txt
    /usr/share/wordlists/rockyou.txt
$ time hashcat -a 0 -m 3200 testhash_bcrypt.txt
    /usr/share/wordlists/rockyou.txt
```

Erwartetes Ergebnis: Der Angriff gegen bcrypt (Modus 3200) dauert deutlich länger als gegen SHA-256, bei identischem Klartext-Passwort. **Erklärung:** Dieser direkte Zeitvergleich ist der praktische Beleg für den in Kapitel 3 erklärten Zusammenhang zwischen Kostenfaktor und Angreifbarkeit. **Troubleshooting:** Kaum Unterschied messbar? Prüfen, ob tatsächlich der korrekte Hashcat-Modus für jedes Format gewählt wurde.

Zusammenfassung

Zusammenfassung

- Hashcat und John the Ripper prüfen dasselbe kryptographische Grundprinzip in hoher Wiederholungszahl: hashen und vergleichen.
- Hashlänge und Präfix erlauben eine erste, schnelle Einordnung des verwendeten Verfahrens.
- GPU-Beschleunigung wirkt besonders stark gegen schnelle, wenig speicherintensive Hashverfahren wie MD5 oder SHA-256.
- Argon2id und bcrypt verlangsamen Cracking-Versuche durch hohen Rechen- und Speicheraufwand gezielt und wirksam.

Kontrollfragen

1. Was unterscheidet Hashcat grundsätzlich von John the Ripper? (leicht)
2. Woran erkennt man einen bcrypt-Hash am Präfix? (leicht)
3. Warum ist GPU-Beschleunigung gegen Argon2id weniger wirksam als gegen MD5? (mittel)
4. Was ist eine Maske im Unterschied zu einem reinen Brute-Force-Angriff? (mittel)
5. Zwei Unternehmen speichern Passwörter jeweils mit SHA-256 und individuellem Salt pro Konto. Reicht der individuelle Salt allein aus, um das Verfahren als sicher für Passwortspeicherung einzustufen? Begründe anhand dieses und des dritten Kapitels. (schwer)

Lösungshinweise: 1. Hashcat ist stark auf schnelle, parallelisierte Passwortprüfung ausgelegt und nutzt häufig GPUs; John the Ripper bietet breite Formatunterstützung und flexible Workflows. Die tatsächlich verfügbare Beschleunigung hängt bei beiden Werkzeugen von Build, Hashformat und Hardware ab, nicht von einer starren GPU-/CPU-Zuordnung. 2. Am Präfix `$2a$`, `$2b$` oder `$2y$`. 3. Weil Argon2id absichtlich hohen Speicherbedarf pro Berechnung nutzt, der sich auf einer GPU nicht beliebig parallelisieren lässt. 4. Eine Maske grenzt den durchprobierten Raum gezielt auf ein plausibles Muster ein, statt den gesamten Zeichenraum ohne Einschränkung zu durchsuchen. 5. Nein, der Salt allein verhindert nur Rainbow-Table-Angriffe; SHA-256 bleibt trotz individuellem Salt ein für Geschwindigkeit optimiertes, also für Passwortspeicherung grundsätzlich ungeeignetes Verfahren, wie bereits in Kapitel 3 erklärt.

Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] erklären kannst, warum Hashcat und John the Ripper erst durch kryptographisches Grundwissen verständlich werden,
- [ ] einen unbekannten Hash anhand von Länge und Präfix identifiziert hast,
- [ ] denselben Klartext gegen ein schnelles und ein langsames Hashverfahren getestet hast,
- [ ] erklären kannst, warum ein individueller Salt allein kein sicheres Passwort-Hashverfahren garantiert.

Kapitel 11: Moderne kryptographische Herausforderungen

Lernziele

- Du erklärst, warum Quantencomputer RSA und ECC gefährden, Hashfunktionen aber kaum.
- Du ordnest ML-KEM, ML-DSA und SLH-DSA ihrem jeweiligen Zweck zu.
- Du erklärst, was kryptographische Agilität bedeutet und warum sie heute vorbereitet wird.

Quantencomputer als neue Bedrohung

Ein **Quantencomputer** nutzt quantenmechanische Effekte, um bestimmte mathematische Probleme grundlegend anders zu lösen als ein klassischer Computer. Für die in diesem Buch behandelte Kryptographie sind zwei Algorithmen entscheidend.

Shors Algorithmus

Shors Algorithmus, 1994 veröffentlicht, würde auf einem ausreichend leistungsfähigen Quantencomputer die Primfaktorzerlegung (Grundlage von RSA, Kapitel 5) und das diskrete Logarithmusproblem (Grundlage von Diffie-Hellman und ECC, Kapitel 5) in praktikabler Zeit lösen. Ein solcher, ausreichend leistungsfähiger Quantencomputer existiert nach aktuellem, öffentlichem Kenntnisstand noch nicht, die Entwicklung wird aber aktiv vorangetrieben.

Grovers Algorithmus

Grovers Algorithmus beschleunigt die Suche in einem unsortierten Raum, etwa das Durchsuchen aller möglichen Hashfunktions-Eingaben, aber nur quadratisch, nicht exponentiell. Eine Hashfunktion mit 256 Bit Ausgabelänge (Kapitel 2) bliebe gegenüber Grovers Algorithmus effektiv auf einem

Sicherheitsniveau von 128 Bit, was weiterhin als praktisch sicher gilt. Hashfunktionen sind von der Quantenbedrohung also deutlich weniger betroffen als asymmetrische Verfahren.

> **Gut zu wissen**
>
> Dieser Unterschied erklärt, warum die Post-Quantum-Migration sich vor allem auf asymmetrische Verfahren (RSA, Diffie-Hellman, ECC) konzentriert, kaum aber auf Hashfunktionen oder symmetrische Verschlüsselung wie AES.

Post-Quantum Cryptography

Post-Quantum Cryptography (PQC) bezeichnet Verfahren, die auf mathematischen Problemen beruhen, für die auch mit Shors Algorithmus bislang kein effizienter Lösungsweg bekannt ist, etwa gitterbasierte oder hashbasierte Konstruktionen.

ML-KEM, ML-DSA und SLH-DSA

Die US-Standardisierungsbehörde NIST hat 2024 die ersten Post-Quantum-Standards veröffentlicht.

Standard	FIPS-Nummer	Zweck
ML-KEM	FIPS 203	Schlüsseleinkapselung, Ersatz für RSA/ECC-basierten Schlüsselaustausch (vergleiche Kapitel 5)
ML-DSA	FIPS 204	digitale Signaturen, gitterbasiert (vergleiche Kapitel 6)
SLH-DSA	FIPS 205	digitale Signaturen, hashbasiert, konservativerer, aber langsamerer Ansatz als ML-DSA

NIST-Standardisierung

Diese drei Standards sind das Ergebnis eines mehrjährigen, öffentlichen Auswahlverfahrens, in dem weltweit eingereichte Verfahren von der Fachwelt auf Sicherheit und Praxistauglichkeit geprüft wurden, ganz im Sinne des in Kapitel 1 erwähnten Kerckhoffs'schen Prinzips: öffentlich geprüft statt geheim gehalten.

Kryptographische Agilität

Kryptographische Agilität bezeichnet die Fähigkeit eines Systems, ein kryptographisches Verfahren auszutauschen, ohne die gesamte Architektur neu entwerfen zu müssen. Systeme, die heute bereits agil gebaut werden, etwa durch austauschbare Cipher Suites in TLS (Kapitel 8), lassen sich leichter auf Post-Quantum-Verfahren umstellen, sobald das nötig wird.

Zukunft der Verschlüsselung und Migration bestehender Systeme

Eine vollständige Migration auf Post-Quantum-Verfahren betrifft vor allem langlebige Geheimnisse: Daten, die heute verschlüsselt abgefangen und gespeichert werden könnten, um sie erst in einigen Jahren mit einem dann existierenden Quantencomputer zu entschlüsseln, ein Szenario, das als „Harvest now, decrypt later" bekannt ist. Kurzlebige Sitzungsschlüssel (Kapitel 8) sind von diesem Szenario weniger betroffen als langfristig vertrauliche Daten.

> **WARNUNG – Sicherheitsrisiko**
>
> „Harvest now, decrypt later" bedeutet, dass bereits heute verschlüsselt übertragene, besonders langfristig sensible Daten ein zukünftiges Risiko darstellen können, selbst wenn aktuell kein Quantencomputer existiert, der sie entschlüsseln könnte.

Zusammenfassung

Zusammenfassung

- Shors Algorithmus würde RSA, Diffie-Hellman und ECC auf einem ausreichend leistungsfähigen Quantencomputer brechen, existiert praktisch bislang aber nicht.
- Grovers Algorithmus schwächt Hashfunktionen nur moderat, nicht grundlegend.
- ML-KEM, ML-DSA und SLH-DSA sind die 2024 von NIST veröffentlichten ersten Post-Quantum-Standards.
- Kryptographische Agilität erleichtert den künftigen Wechsel zu Post-Quantum-Verfahren.

Kontrollfragen

1. Welcher Algorithmus bedroht RSA und ECC auf einem Quantencomputer? (leicht)
2. Nenne die drei 2024 von NIST veröffentlichten Post-Quantum-Standards. (leicht)
3. Warum sind Hashfunktionen von der Quantenbedrohung weniger betroffen als RSA? (mittel)
4. Was bedeutet kryptographische Agilität? (mittel)
5. Erkläre das Szenario „Harvest now, decrypt later" und nenne eine Art von Daten, für die es besonders relevant ist. (schwer)

Lösungshinweise: 1. Shors Algorithmus. 2. ML-KEM, ML-DSA und SLH-DSA. 3. Weil Grovers Algorithmus die effektive Sicherheit nur quadratisch statt exponentiell verringert, eine 256-Bit-Hashfunktion bleibt damit bei effektiv 128 Bit, was weiterhin sicher gilt. 4. Die Fähigkeit eines Systems, ein kryptographisches Verfahren auszutauschen, ohne die gesamte Architektur neu zu entwerfen. 5. Ein Angreifer zeichnet heute verschlüsselten Datenverkehr auf, um ihn erst in der Zukunft mit einem dann leistungsfähigeren Quantencomputer zu entschlüsseln; besonders relevant für Daten mit sehr langer Vertraulichkeitsanforderung, etwa medizinische Akten oder Staatsgeheimnisse.

Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

[ ] erklären kannst, warum Quantencomputer asymmetrische Verfahren stärker bedrohen als Hashfunktionen,

[ ] ML-KEM, ML-DSA und SLH-DSA ihrem jeweiligen Zweck zuordnest,

[ ] kryptographische Agilität in eigenen Worten erklären kannst,

[ ] das Szenario „Harvest now, decrypt later" an einem eigenen Beispiel erläutern kannst.

Kapitel 12: Kryptographische Fehler und Sicherheitsrisiken

Lernziele

- Du erkennst die häufigsten Implementierungsfehler über alle vorherigen Kapitel hinweg.
- Du unterscheidest einen Algorithmusfehler von einem Implementierungsfehler.
- Du kannst zu jedem in diesem Kapitel genannten Fehler die passende Gegenmaßnahme aus einem früheren Kapitel benennen.

Eine Zusammenschau statt neuer Theorie

Dieses Kapitel führt keine neuen Verfahren ein, sondern bündelt die in den vorherigen Kapiteln bereits einzeln genannten Risiken zu einer Gesamtübersicht, damit sie als Prüfliste nutzbar wird.

Schwache Schlüssel und unsichere Zufallszahlengeneratoren

Ein kryptographischer Schlüssel ist nur so sicher wie der Zufall, aus dem er erzeugt wurde. Ein **unsicherer Zufallszahlengenerator**, etwa einer, der auf einem vorhersehbaren Startwert wie der aktuellen Systemzeit basiert, kann dazu führen, dass scheinbar zufällige Schlüssel tatsächlich erratbar sind. Betriebssysteme stellen dafür eigene, kryptographisch geprüfte Quellen bereit, unter Linux etwa `/dev/urandom`.

Wiederverwendung von Nonces

Kapitel 4 hat erklärt, dass eine Nonce pro Verschlüsselung einzigartig sein muss. Wird derselbe Nonce versehentlich zweimal mit demselself Schlüssel verwendet, etwa bei AES-GCM, kann das die Vertraulichkeit und teilweise sogar die Integritätssicherung des gesamten Verfahrens untergraben, abhängig vom konkreten Modus.

Veraltete Algorithmen und unsichere Hashverfahren

MD5, SHA-1 (Kapitel 2) und SSL beziehungsweise frühe TLS-Versionen (Kapitel 8) gehören zu den am häufigsten noch in Altsystemen anzutreffenden, veralteten Verfahren.

Zertifikatsfehler, Man-in-the-Middle und Downgrade-Angriffe

Ein **Man-in-the-Middle-Angriff** schaltet sich unbemerkt zwischen zwei Kommunikationspartner; eine korrekt geprüfte Zertifikatskette (Kapitel 7) soll genau das verhindern. Ein **Downgrade-Angriff** (Kapitel 8) drängt eine Verbindung gezielt auf eine schwächere, noch unterstützte Verfahrensversion.

Fehlkonfigurationen und schlechte Schlüsselverwaltung

Ein im Quellcode fest hinterlegter Schlüssel, ein nicht rotierter, über Jahre unveränderter Schlüssel, und eine fehlende Trennung zwischen Test- und Produktivschlüsseln gehören zu den verbreitetsten organisatorischen Fehlern, unabhängig davon, wie korrekt der zugrunde liegende Algorithmus selbst implementiert ist.

Unsichere Implementierungen

Selbst ein mathematisch korrektes Verfahren kann durch eine fehlerhafte Implementierung unsicher werden, etwa durch **Timing-Angriffe**, bei denen geringfügige Laufzeitunterschiede einer Operation Rückschlüsse auf geheime Daten erlauben, oder durch unzureichend geschützten Arbeitsspeicher, aus dem Schlüssel nachträglich ausgelesen werden können.

Übersicht: Fehler und ihre Gegenmaßnahme

Fehler	Gegenmaßnahme aus einem früheren Kapitel
Passwörter mit SHA-256 gehasht	Argon2id oder bcrypt verwenden (Kapitel 3)
ECB-Modus für strukturierte Daten	GCM oder CBC mit korrektem IV verwenden (Kapitel 4)
Fest hinterlegter Schlüssel im Quellcode	Schlüsseltresor oder HSM verwenden (Kapitel 4, 7)
Veraltetes TLS 1.0 weiterhin aktiv	ausschließlich TLS 1.2 und 1.3 zulassen (Kapitel 8)
Unvollständige Zertifikatskette	Intermediate-Zertifikat korrekt mit ausliefern (Kapitel 7)
Wiederverwendeter Nonce	pro Verschlüsselung neuen, einzigartigen Nonce erzeugen (Kapitel 4)

Typische Fehlannahmen

„Verschlüsselung allein reicht" übersieht, dass auch Integrität und Authentizität eigene Mechanismen brauchen (Kapitel 6). „Ein langes Passwort reicht auch mit schwachem Hashverfahren" übersieht, dass die Geschwindigkeit des Hashverfahrens selbst über die Angreifbarkeit mitentscheidet (Kapitel 3 und 10). „HTTPS bedeutet automatisch Ende-zu-Ende-Verschlüsselung" übersieht den in Kapitel 9 erklärten Unterschied zwischen Transport- und Ende-zu-Ende-Verschlüsselung.

Maßnahmen zur Absicherung

Regelmäßige Schlüsselrotation, konsequente Nutzung aktueller, öffentlich geprüfter Standardbibliotheken statt eigener kryptographischer Implementierungen, und eine jährliche Überprüfung eingesetzter Algorithmen gegen aktuelle NIST- und IETF-Empfehlungen bilden zusammen eine solide organisatorische Grundlage.

> **Tipp**
>
> Implementiere kryptographische Verfahren nahezu nie selbst von Grund auf. Nutze stattdessen etablierte, öffentlich geprüfte Bibliotheken wie OpenSSL; selbst kleine Implementierungsfehler in einer eigenen Umsetzung können ein ansonsten sicheres Verfahren vollständig untergraben.

Zusammenfassung

> **Zusammenfassung**
>
> • Die meisten kryptographischen Sicherheitsvorfälle entstehen nicht durch gebrochene Algorithmen, sondern durch Implementierungs- und Konfigurationsfehler.
> • Schwache Zufallszahlengeneratoren und wiederverwendete Nonces untergraben selbst mathematisch korrekte Verfahren.
> • Zertifikatsfehler, Man-in-the-Middle- und Downgrade-Angriffe zielen auf die Vertrauenskette, nicht auf den Algorithmus selbst.
> • Etablierte, öffentlich geprüfte Bibliotheken sind eigenen kryptographischen Implementierungen fast immer vorzuziehen.

Kontrollfragen

1. Was unterscheidet einen Algorithmusfehler von einem Implementierungsfehler? (leicht)

2. Was ist ein Timing-Angriff? (leicht)

3. Warum kann ein wiederverwendeter Nonce bei AES-GCM besonders schwerwiegend sein? (mittel)

4. Welche Fehlannahme steckt hinter dem Satz „HTTPS bedeutet automatisch Ende-zu-Ende-Verschlüsselung"? (mittel)

5. Ein Unternehmen verwendet AES-256, ein nachweislich sicheres Verfahren, verschlüsselt aber jede Nachricht mit demselben, fest im Quellcode hinterlegten Schlüssel und Initialisierungsvektor. Ist das System sicher? Begründe anhand mehrerer Kapitel. (schwer)

Lösungshinweise: 1. Ein Algorithmusfehler liegt im mathematischen Verfahren selbst, ein Implementierungsfehler in seiner konkreten,

fehlerhaften Umsetzung. 2. Ein Angriff, der aus geringfügigen Laufzeitunterschieden einer Operation Rückschlüsse auf geheime Daten zieht. 3. Weil die Kombination aus Schlüssel und Nonce bei GCM sowohl Vertraulichkeit als auch die eingebaute Integritätsprüfung absichert; eine Wiederverwendung kann beides gleichzeitig untergraben. 4. Dass HTTPS lediglich die Übertragungsstrecke schützt, der Diensteanbieter selbst aber weiterhin Zugriff auf den Klartext haben kann, wie in Kapitel 9 erklärt. 5. Nein, trotz des sicheren Algorithmus AES-256 ist das System unsicher: Ein fest hinterlegter Schlüssel kann aus dem Quellcode extrahiert werden (Kapitel 4), und ein wiederverwendeter Initialisierungsvektor macht identische Klartextabschnitte über mehrere Nachrichten hinweg erkennbar (Kapitel 4 und dieses Kapitel).

Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] die häufigsten Implementierungsfehler aus den vorherigen Kapiteln wiedererkennst,
- [ ] zu jedem genannten Fehler die passende Gegenmaßnahme benennen kannst,
- [ ] einen Algorithmusfehler von einem Implementierungsfehler unterscheiden kannst,
- [ ] erklären kannst, warum eigene kryptographische Implementierungen vermieden werden sollten.

Kapitel 13: Praxisprojekt – Eine sichere kryptographische Testumgebung

Lernziele

- Du baust eine vollständige, eigene Testumgebung für kryptographische Übungen auf.
- Du wendest alle in diesem Buch behandelten Verfahren in einem einzigen, zusammenhängenden Projekt praktisch an.
- Du erstellst eine eigene, kurze Sicherheitsbewertung deiner Testumgebung.

Projektziel

Dieses Kapitel führt zwölf Einzelaufgaben zu einem zusammenhängenden Projekt zusammen: eine eigene Testumgebung, in der du nacheinander Hashfunktionen, symmetrische und asymmetrische Verschlüsselung, Signaturen, eine eigene PKI und TLS praktisch durchläufst, abschließend ergänzt um einen Passwort-Hash-Test und eine eigene Dokumentation.

Laborumgebung

Komponente	Zweck
Linux-VM	Haupt-Arbeitsumgebung für alle zwölf Aufgaben
Windows-Client oder zweite Linux-VM	für Aufgabe 7 (HTTPS-Verbindung aus Sicht eines Clients)
OpenSSL	Aufgaben 1 und 3 bis 6
Python 3 mit dem Paket `cryptography`	Aufgabe 2 (AES-256-GCM, da `openssl enc` dafür nicht geeignet ist)

Komponente	Zweck
Wireshark	Aufgabe 8
Hashcat, John the Ripper	Aufgaben 9 und 10

> **Rechtlicher Hinweis**
>
> Dieses Projekt findet ausschließlich auf eigenen, selbst kontrollierten Testsystemen statt. Keine der hier erzeugten Testzertifikate, Schlüssel oder Konfigurationen ist für den produktiven Einsatz vorgesehen.

Aufgabe 1: SHA-256-Hashes erzeugen und vergleichen

Lernziel: die Grundlage aus Kapitel 2 im Projektkontext wiederholen. **Voraussetzungen:** Linux-VM mit Coreutils. **Werkzeuge:** `sha256sum`. **Anleitung:**

Befehle zu Aufgabe 1

```
$ echo -n "Projektstart" | sha256sum
$ echo -n "Projektstart!" | sha256sum
```

Erwartete Ausgabe: zwei völlig unterschiedliche 64-stellige Hexadezimalwerte. **Interpretation:** bestätigt den Avalanche-Effekt aus Kapitel 2 als Ausgangspunkt für das gesamte Projekt. **Typischer Fehler:** vergessenes `-n` bei `echo`, wodurch ein zusätzliches Newline-Zeichen den Hashwert verfälscht. **Troubleshooting:** Ergebnis weicht von früheren Übungen ab? Exakte Eingabe, einschließlich Groß-/Kleinschreibung, erneut prüfen. **Sicherheitsbetrachtung:** keine, reine Demonstration. **Kontrollfrage:** Warum verändert ein einziges zusätzliches Zeichen den gesamten Hashwert?

Aufgabe 2: Eine Datei mit AES verschlüsseln

Lernziel: Kapitel 4 im Projektkontext anwenden, einschließlich eines erkannten Manipulationsversuchs. **Voraussetzungen:** Python 3 mit dem Paket `cryptography`; die beiden Skripte `aesgcm_verschluesseln.py`

und `aesgcm_entschluesseln.py` aus Kapitel 4, Übung A und B, kopiert in das Projektverzeichnis. **Anleitung:** Passe in beiden kopierten Skripten die Dateinamen an: `Path("geheim.txt")` wird zu `Path("projekt.txt")`, `Path("geheim.enc")` zu `Path("projekt.enc")`. Führe danach aus:

Befehle zu Aufgabe 2

```
$ echo "Projektgeheimnis" > projekt.txt
$ python3 aesgcm_verschluesseln.py
$ python3 aesgcm_entschluesseln.py
$ diff projekt.txt entschluesselt.txt
```

Erwartete Ausgabe: eine binäre, nicht lesbare Datei `projekt.enc`; nach der Entschlüsselung eine zu `projekt.txt` identische Datei. **Interpretation:** AES-256-GCM liefert gleichzeitig Vertraulichkeit und Integritätsschutz (Kapitel 4); `openssl enc` unterstützt GCM nicht, deshalb kommen dieselben Python-Skripte wie in Kapitel 4, Übung A und B zum Einsatz, mit angepassten Dateinamen. **Typischer Fehler:** `openssl enc -aes-256-gcm` anstelle der Python-Skripte zu verwenden (schlägt grundsätzlich fehl); oder die Dateinamen in den kopierten Skripten nicht anzupassen, sodass weiterhin `geheim.txt` statt `projekt.txt` verarbeitet wird. **Troubleshooting:** `FileNotFoundError: geheim.txt`? Die Dateinamen in den kopierten Skripten wurden nicht angepasst; siehe oben. Ansonsten siehe Kapitel 4, Übung A und B. **Sicherheitsbetrachtung:** Verändere versuchsweise ein Byte in `projekt.enc` und führe die Entschlüsselung erneut aus; die `InvalidTag`-Ausnahme belegt den eingebauten Integritätsschutz von GCM. **Kontrollfrage:** Welchen zusätzlichen Vorteil bietet GCM gegenüber reinem CBC?

Aufgabe 3: Ein RSA-Schlüsselpaar erstellen

Lernziel: Kapitel 5 im Projektkontext anwenden. **Anleitung:**

Befehle zu Aufgabe 3

```
$ openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 \
    -out projekt_private.pem
$ openssl rsa -pubout -in projekt_private.pem -out
    projekt_public.pem
```

Erwartete Ausgabe: zwei PEM-Dateien, privater und öffentlicher Schlüssel. **Interpretation:** dieses Schlüsselpaar wird in Aufgabe 4 für eine Signatur weiterverwendet. **Typischer Fehler:** versehentliche Weitergabe von `projekt_private.pem` statt des öffentlichen Teils. **Troubleshooting:** siehe Kapitel 5, Übung A. **Sicherheitsbetrachtung:** Schütze `projekt_private.pem` von Anfang an wie ein echtes Geheimnis: Datei mit `chmod 600 projekt_private.pem` auf nur den eigenen Benutzer beschränken, und für ein echtes Projekt zusätzlich mit einem eigenen Passwort verschlüsseln (`-aes256` beim Erzeugen). Solltest du dieses Schlüsselpaar später auch zur RSA-Verschlüsselung nutzen, verwende ausdrücklich OAEP (Kapitel 5, Hinweis zu Beispiel 5.3). **Kontrollfrage:** Welcher der beiden Schlüssel darf weitergegeben werden?

Aufgabe 4: Eine digitale Signatur erzeugen und prüfen

Lernziel: Kapitel 6 im Projektkontext anwenden. **Anleitung:**

Befehle zu Aufgabe 4

```
$ openssl dgst -sha256 -sign projekt_private.pem \
    -out projekt.sig projekt.txt
$ openssl dgst -sha256 -verify projekt_public.pem \
    -signature projekt.sig projekt.txt
```

Erwartete Ausgabe: `Verified OK`. **Interpretation:** bestätigt Integrität und Authentizität der Datei `projekt.txt` aus Aufgabe 2. **Typischer Fehler:** Verwechslung von öffentlichem und privatem Schlüssel beim Signieren. **Troubleshooting:** siehe Kapitel 6, Übung B. **Sicherheitsbetrachtung:** keine zusätzliche, über Kapitel 6 hinausgehende. **Kontrollfrage:** Welche zwei Sicherheitsziele bestätigt diese Signatur?

Aufgabe 5: Eine eigene Test-CA einrichten

Lernziel: Kapitel 7 im Projektkontext anwenden, mit eindeutigen CA-Erweiterungen. **Anleitung:** Übernimm die Befehle aus Kapitel 7, Übung A, aber ersetze darin jedes `ca.key` durch `projekt_ca.key` und jedes `ca.crt` durch `projekt_ca.crt`; den Namen setzt du auf `/CN=Projekt-CA`. Ergänze beim Signieren künftiger Zertifikate stets eine

Extensions-Datei wie in Kapitel 7, Übung B, damit ausgestellte Zertifikate eindeutig als Server- statt als CA-Zertifikate gekennzeichnet sind. **Erwartete Ausgabe:** `projekt_ca.key` und `projekt_ca.crt`. **Interpretation:** diese CA signiert im nächsten Schritt das Testzertifikat. **Typischer Fehler:** zu kurze Schlüssellänge für eine Root CA. **Troubleshooting:** siehe Kapitel 7, Übung A. **Sicherheitsbetrachtung:** `projekt_ca.key` ist der sensibelste Schlüssel dieses gesamten Projekts, verdient besonderen Schutz selbst im Testlabor (`chmod 600`), und sollte getrennt von allen Serverschlüsseln aufbewahrt werden. **Kontrollfrage:** Warum ist der private Schlüssel einer CA besonders schützenswert?

Aufgabe 6: Ein TLS-Testzertifikat erstellen

Lernziel: Kapitel 7 vollständig durchlaufen und ein tatsächlich HTTPS-taugliches Zertifikat erzeugen. **Anleitung:** Übernimm die Befehle aus Kapitel 7, Übung B und C, mit folgenden Ersetzungen: `server.key`/`server.csr`/`server.crt` bleiben wie dort gezeigt, `ca.crt`/`ca.key` werden zu `projekt_ca.crt`/`projekt_ca.key` aus Aufgabe 5, das CSR-Subject wird `/CN=projekt.local`, und die Extensions-Datei heißt `projekt_server_ext.cnf` mit dem Inhalt `basicConstraints = critical, CA:FALSE`, passendem `keyUsage`, `extendedKeyUsage = serverAuth` und `subjectAltName = DNS:projekt.local`. **Erwartete Ausgabe:** ein von `projekt_ca.crt` signiertes Zertifikat `server.crt` mit SAN, bestätigt durch `openssl verify -CAfile projekt_ca.crt server.crt` und zusätzlich durch `openssl verify -CAfile projekt_ca.crt -verify_hostname projekt.local server.crt`. **Interpretation:** dieses Zertifikat wird in Aufgabe 7 für eine lokale HTTPS-Verbindung genutzt. **Typischer Fehler:** die Extensions-Datei beim Signieren zu vergessen (`-extfile` fehlt); das Zertifikat ist dann zwar gültig signiert, aber ohne SAN für eine normale HTTPS-Prüfung ungeeignet. **Troubleshooting:** siehe Kapitel 7, Übung B und C. **Sicherheitsbetrachtung:** Ob eine Zertifikatswarnung erscheint, hängt davon ab, ob der prüfende Client `projekt_ca.crt` kennt. Ist die Test-CA dort nicht als vertrauenswürdig hinterlegt (etwa ein normaler Browser ohne manuellen Import), ist eine Warnung zu erwarten und kein Fehler. Wird dagegen, wie in Aufgabe 7, gezielt mit `-CAfile projekt_ca.crt` geprüft und ist das Zertifikat korrekt ausgestellt, sollte die Prüfung ohne Warnung erfolgreich verlaufen. Ein passender Subject

Alternative Name allein macht ein Zertifikat dabei noch nicht vertrauenswürdig: Ohne eine als vertrauenswürdig bekannte, ausstellende CA bleibt die Verbindung ungeprüft, unabhängig davon, wie korrekt der SAN gesetzt ist. **Kontrollfrage:** Welche zwei Angaben prüft ein Client in diesem Zertifikat zuerst?

Aufgabe 7: Eine HTTPS-Verbindung untersuchen

Lernziel: Kapitel 8 im Projektkontext anwenden, mit vollständiger Zertifikats- und Hostnamenprüfung statt nur der ausgehandelten Version. **Voraussetzungen:** ein einfacher, lokaler Webserver mit dem Zertifikat aus Aufgabe 6. **Anleitung:** siehe Kapitel 8, Übung A und B, gegen den eigenen lokalen Server statt einer öffentlichen Webseite, mit folgendem vollständigen Aufruf:

Befehl zu Aufgabe 7

```
$ openssl s_client \
    -connect projekt.local:443 \
    -servername projekt.local \
    -CAfile projekt_ca.crt \
    -verify_return_error \
    -verify_hostname projekt.local
```

Erwartete Ausgabe: die ausgehandelte TLS-Version, das selbst ausgestellte Zertifikat, und eine erfolgreiche Hostnamenprüfung ohne Verifikationsfehler. **Erklärung:** `-CAfile projekt_ca.crt` legt fest, welcher Zertifizierungsstelle diese eine Verbindung vertrauen soll (vergleiche Kapitel 7); `-servername` setzt SNI (Server Name Indication), damit der Server bei mehreren gehosteten Namen das richtige Zertifikat ausliefert; `-verify_hostname` prüft zusätzlich, ob `projekt.local` tatsächlich im SAN des präsentierten Zertifikats steht; `-verify_return_error` lässt die Verbindung bei einem tatsächlichen Prüffehler abbrechen, statt nur eine Warnung auszugeben und trotzdem fortzufahren (Kapitel 8, Übung A). **Interpretation:** derselbe Handschlag wie in Diagramm 8.1, diesmal vollständig selbst kontrolliert, einschließlich des in Kapitel 7 ergänzten Hostnamen-Schritts. **Typischer Fehler:** Server nutzt versehentlich nur HTTP statt HTTPS; oder die Kettenprüfung wird fälschlich mit einer vollständigen Hostnamenprüfung gleichgesetzt (Kapitel 7). **Troubleshooting:** siehe Kapitel 8, Übung A. **Sicherheitsbetrachtung:** siehe die Klarstellung zur

Zertifikatswarnung am Ende von Aufgabe 6. **Kontrollfrage:** Welche TLS-Version wurde ausgehandelt, und warum genau diese?

Aufgabe 8: TLS-Verkehr mit Wireshark analysieren

Lernziel: Kapitel 8 und 9 praktisch verbinden. **Anleitung:** siehe Kapitel 8, Übung C, gegen den eigenen lokalen Server aus Aufgabe 7. **Erwartete Ausgabe:** sichtbares ClientHello und ServerHello im Klartext, danach ausschließlich verschlüsselte Pakete vom Typ „Application Data", auch für die restlichen Handshake-Nachrichten des Servers (Diagramm 8.1). **Interpretation:** bestätigt praktisch, dass bei TLS 1.3 nur ClientHello und ServerHello unverschlüsselt sind und bereits der restliche Handschlag verschlüsselt abläuft, nicht erst die eigentliche Anwendungskommunikation. **Typischer Fehler:** Mitschnitt an der falschen Netzwerkschnittstelle; ein sichtbares, lesbares Certificate-Paket nach ServerHello fälschlich für normal halten, statt es als Hinweis auf eine über TLS 1.2 statt TLS 1.3 ausgehandelte Verbindung zu erkennen. **Troubleshooting:** siehe Wireshark-Handbuch dieser Fachbibliothek, Kapitel 2. **Sicherheitsbetrachtung:** keine zusätzliche. **Kontrollfrage:** Ab welchem Punkt im Mitschnitt lässt sich der Inhalt nicht mehr lesen?

Aufgabe 9: Einen eigenen Passwort-Hash untersuchen

Lernziel: Kapitel 3 und 10 im Projektkontext anwenden. **Anleitung:** siehe Kapitel 3, Übung A, mit einem selbst gewählten Projektpasswort. **Erwartete Ausgabe:** ein bcrypt-Hash im Format `$2b$...`. **Interpretation:** Grundlage für den Vergleich in Aufgabe 10. **Typischer Fehler:** keiner spezifisch. **Troubleshooting:** siehe Kapitel 3, Übung A. **Sicherheitsbetrachtung:** keine zusätzliche. **Kontrollfrage:** Welche Informationen stecken im bcrypt-Präfix selbst?

Aufgabe 10: Unterschiedliche Passwort-Hashverfahren vergleichen

Lernziel: Kapitel 10 abschließend im Projektkontext anwenden. **Anleitung:** siehe Kapitel 10, Übung D, mit dem Projektpasswort aus Aufgabe 9. **Erwartete Ausgabe:** ein messbarer Zeitunterschied zwischen einem

SHA-256- und einem bcrypt-Angriff auf der eigenen, dokumentierten Hardware. **Interpretation:** schließt den Bogen vom theoretischen Kapitel 3 zum praktischen Kapitel 10. Notiere dabei Hashcat-Version, Hashmodus, CPU/GPU-Modell und die verwendeten bcrypt-Kostenparameter; nur mit diesen Angaben lässt sich der gemessene Faktor später nachvollziehen oder mit anderer Hardware vergleichen. **Typischer Fehler:** falscher Hashcat-Modus für das jeweilige Format; der gemessene Faktor wird fälschlich als allgemeingültige Aussage statt als Ergebnis der eigenen, konkreten Testumgebung behandelt. **Troubleshooting:** siehe Kapitel 10, Übung D. **Sicherheitsbetrachtung:** ausschließlich gegen das selbst erzeugte Testpasswort, niemals gegen fremde Daten. **Kontrollfrage:** Welcher ungefähre Faktor ergab sich auf deiner eigenen Hardware, und welche Parameter müsstest du nennen, damit jemand anderes diesen Wert nachvollziehen kann?

Aufgabe 11: Ergebnisse dokumentieren

Lernziel: alle zehn vorherigen Aufgaben in einer zusammenhängenden Übersicht festhalten. **Anleitung:** Erstelle eine Tabelle mit einer Zeile je Aufgabe, mit Spalten für Werkzeug samt Versionsnummer, verwendete Parameter, Ergebnis, aufgetretene Fehlermeldungen und Datum. **Erwartete Ausgabe:** eine vollständige, zehnzeilige Dokumentation, aus der sich jede Aufgabe auf einem anderen System mit denselben Versionen und Parametern nachvollziehen ließe. **Interpretation:** diese Dokumentation ist die Grundlage für Aufgabe 12. **Typischer Fehler:** unvollständige Dokumentation einzelner Aufgaben. **Troubleshooting:** jede fehlende Zeile anhand der jeweiligen Aufgabe in diesem Kapitel ergänzen. **Sicherheitsbetrachtung:** keine. **Kontrollfrage:** Welche Aufgabe baute unmittelbar auf einer anderen auf, und warum?

Aufgabe 12: Eine Sicherheitsbewertung erstellen

Lernziel: das gesamte Projekt kritisch, aus Sicht von Kapitel 12, bewerten. **Anleitung:** Prüfe die eigene Dokumentation aus Aufgabe 11 gegen die Fehlerliste aus Kapitel 12: Wurden Testschlüssel und Testpasswörter angemessen behandelt? Wurde mit modernen Verfahren gearbeitet? **Erwartete Ausgabe:** eine kurze, schriftliche Bewertung von höchstens einer Seite. **Interpretation:** schließt das Projekt mit genau der kritischen

Prüfhaltung ab, die auch bei echten, produktiven Systemen angemessen ist. **Typischer Fehler:** reine Erfolgsmeldung ohne kritische Prüfung tatsächlicher Schwachstellen im eigenen Testaufbau. **Troubleshooting:** Kapitel 12 erneut konsultieren, falls die Bewertung zu oberflächlich ausfällt. **Sicherheitsbetrachtung:** diese Aufgabe ist selbst die Sicherheitsbetrachtung des gesamten Projekts. **Kontrollfrage:** Welche einzelne Verbesserung würdest du an deiner eigenen Testumgebung zuerst vornehmen?

Zusammenfassung

> **Zusammenfassung**
>
> • Dieses Projekt verbindet Hashfunktionen, symmetrische und asymmetrische Verschlüsselung, Signaturen, PKI und TLS zu einem einzigen, zusammenhängenden Ablauf.
> • Jede Aufgabe baut auf einem bereits in einem früheren Kapitel erklärten Verfahren auf, ohne es erneut herzuleiten.
> • Eine abschließende, kritische Sicherheitsbewertung gehört zu jedem kryptographischen Projekt, auch im eigenen Testlabor.

Kapitel 14: Zusammenfassung und Wissenstest

Lernziele

- Du ordnest jedes in diesem Buch behandelte Verfahren einem der vier Sicherheitsziele zu.
- Du bestehst den abschließenden Wissenstest zu allen dreizehn vorherigen Kapiteln.
- Du nutzt die Entscheidungshilfen dieses Kapitels als Nachschlagewerk für künftige Projekte.

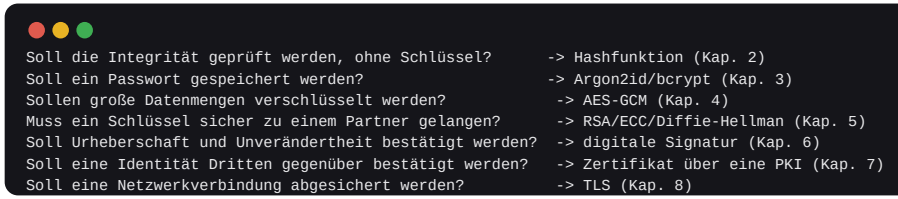
Die wichtigsten kryptographischen Verfahren im Überblick

Verfahren	Kapitel	Hauptzweck
SHA-256 / SHA-3	2	Integritätsprüfung, Fingerabdruck von Daten
Argon2id / bcrypt	3	sichere Passwortspeicherung
AES-GCM	4	symmetrische Verschlüsselung großer Datenmengen
RSA mit OAEP	5	asymmetrische Verschlüsselung
ECDH / X25519	5	Schlüsselaustausch
ECDSA / Ed25519	6	digitale Signaturen
X.509-Zertifikate	7	Identitätsbestätigung über eine PKI

Verfahren	Kapitel	Hauptzweck
TLS 1.3	8	Absicherung von Netzwerkverbindungen
ML-KEM / ML-DSA	11	Post-Quantum-Schutz künftiger Systeme

Entscheidungshilfe: Welches Verfahren für welchen Zweck?

Diagramm 14.1: Entscheidungsbaum (stark vereinfacht)



Multiple-Choice-Fragen

Frage 1: Welche Eigenschaft beschreibt, dass eine minimale Eingabeänderung den gesamten Hashwert verändert? a) Kollisionsresistenz b) Avalanche-Effekt c) Forward Secrecy d) Kryptographische Agilität

Frage 2: Welches Verfahren eignet sich für sichere Passwortspeicherung? a) SHA-256 b) MD5 c) Argon2id d) Base64

Frage 3: Was verschlüsselt bei asymmetrischer Verschlüsselung? a) ausschließlich der Private Key b) der Public Key c) ein gemeinsamer, symmetrischer Schlüssel d) keiner der beiden Schlüssel allein

Frage 4: Welche Aussage über digitale Signaturen ist korrekt? a) Sie sichern primär Vertraulichkeit. b) Signieren erfolgt mit dem Public Key. c) Sie sichern Integrität und Authentizität. d) Sie ersetzen vollständig eine PKI.

Frage 5: Was bestätigt eine Root CA in einer PKI? a) die Verschlüsselungsstärke eines Algorithmus b) die Zuordnung eines Public Keys zu einer Identität c) die Geschwindigkeit eines Hashverfahrens d) die Gültigkeit eines Passworts

Frage 6: Wie viele Runden benötigt ein TLS-1.3-Handshake minimal? a) drei b) zwei c) eine Hin- und Rückrunde d) keine, TLS 1.3 verzichtet auf einen Handshake

Frage 7: Was unterscheidet Transportverschlüsselung von Ende-zu-Ende-Verschlüsselung? a) nichts, beide Begriffe meinen dasselbe b) bei Transportverschlüsselung kann ein zwischengeschalteter Server mitlesen c) Ende-zu-Ende-Verschlüsselung nutzt niemals Hybridverschlüsselung d) Transportverschlüsselung ist grundsätzlich unsicher

Frage 8: Warum ist GPU-Beschleunigung gegen Argon2id weniger wirksam als gegen MD5? a) Argon2id ist mathematisch völlig anders aufgebaut als jede andere Hashfunktion b) Argon2id nutzt absichtlich hohen Speicherbedarf, der sich schlecht parallelisieren lässt c) GPUs können Argon2id technisch gar nicht berechnen d) Argon2id verwendet keine Hashfunktion

Frage 9: Welcher Algorithmus würde RSA auf einem ausreichend leistungsfähigen Quantencomputer brechen? a) Grovers Algorithmus b) AES c) Shors Algorithmus d) SHA-3

Frage 10: Was ist ein typischer, in Kapitel 12 genannter Implementierungsfehler? a) die Verwendung von AES-256 b) ein wiederverwendeter Nonce c) die Nutzung von TLS 1.3 d) ein individueller Salt pro Konto

Praktische Verständnisaufgaben

Aufgabe 1: Erkläre einer Person ohne IT-Hintergrund in maximal drei Sätzen, warum ein Passwort gehasht statt verschlüsselt gespeichert wird.

Aufgabe 2: Skizziere aus dem Gedächtnis den TLS-1.3-Handshake aus Diagramm 8.1, ohne das Kapitel erneut nachzuschlagen.

Aufgabe 3: Nenne zu jedem der vier Sicherheitsziele aus Kapitel 1 ein Verfahren aus diesem Buch, das genau dieses Ziel primär erfüllt.

Lösungen mit ausführlichen Erklärungen

Multiple-Choice-Lösungen: 1b, 2c, 3b, 4c, 5b, 6c, 7b, 8b, 9c, 10b.

Aufgabe 1 (Musterlösung): Ein System muss nur prüfen können, ob ein eingegebenes Passwort korrekt ist, nicht das Passwort selbst kennen. Ein Hashwert erlaubt genau diesen Vergleich, lässt sich aber nicht zurückrechnen, selbst wenn ein Angreifer ihn stiehlt. Deshalb reicht ein Hashwert aus, ein vollständig entschlüsselbares Passwort wäre ein unnötiges, zusätzliches Risiko.

Aufgabe 2 (Musterlösung): Der Client sendet ClientHello mit den unterstützten Verfahren und einem Schlüsselaustausch-Wert. Der Server antwortet mit ServerHello und seinem eigenen Schlüsselaustausch-Wert. Daraus leiten beide Seiten dieselben Handshake-Schlüssel ab. Anschließend sendet der Server seine Handshake-Nachrichten bereits verschlüsselt, darunter Certificate, CertificateVerify und Finished. Danach folgen die geschützten Anwendungsdaten.

Aufgabe 3 (Musterlösung): Vertraulichkeit durch symmetrische oder asymmetrische Verschlüsselung (Kapitel 4, 5); Integrität durch Hashfunktionen oder Signaturen (Kapitel 2, 6); Authentizität durch digitale Signaturen oder Zertifikate (Kapitel 6, 7); Verbindlichkeit durch digitale Signaturen (Kapitel 6).

Zusammenfassung

> **Zusammenfassung**
>
> • Jedes in diesem Buch behandelte Verfahren lässt sich einem oder mehreren der vier klassischen Sicherheitsziele aus Kapitel 1 zuordnen.
> • Der Entscheidungsbaum in diesem Kapitel dient als Nachschlagewerk für künftige, eigene Projekte.
> • Wer alle Multiple-Choice-Fragen und Verständnisaufgaben dieses Kapitels ohne Nachschlagen löst, hat die zentralen Inhalte dieses Buches verinnerlicht.

Anhang

OpenSSL-Spickzettel

Spickzettel: Die wichtigsten OpenSSL-Befehle dieses Buches

```
# Hashwerte und Signaturen
openssl dgst -sha256 datei.txt
openssl dgst -sha256 -sign private.pem -out datei.sig datei.txt
openssl dgst -sha256 -verify public.pem -signature datei.sig
    datei.txt

# Symmetrische Verschlüsselung (AES-GCM): openssl enc unterstuetzt
# keine AEAD-Modi wie GCM. Siehe stattdessen die Python-Skripte
# aesgcm_verschluesseln.py / aesgcm_entschluesseln.py aus Kapitel
    4,
# Uebung A und B.

# Asymmetrische Schluessel
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -out
    private.pem
openssl rsa -pubout -in private.pem -out public.pem

# RSA-Verschluesselung mit explizitem OAEP (Kapitel 5, Uebung C)
openssl pkeyutl -encrypt -pubin -inkey public.pem -in datei.txt
    -out datei.enc \
    -pkeyopt rsa_padding_mode:oaep -pkeyopt rsa_oaep_md:sha256
        -pkeyopt rsa_mgf1_md:sha256

# Zertifikate und PKI (vollstaendiger Ablauf, Kapitel 7)
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -out
    ca.key
openssl req -new -x509 -key ca.key -days 3650 -out ca.crt -subj
    "/CN=Test-CA"
openssl req -new -key server.key -out server.csr -subj
    "/CN=host.local"
openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key
    -CAcreateserial \
    -out server.crt -extfile server_ext.cnf
openssl verify -CAfile ca.crt server.crt
openssl verify -CAfile ca.crt -verify_hostname host.local
    server.crt
openssl x509 -in server.crt -text -noout
```

```
# server_ext.cnf muss mindestens enthalten:
#   basicConstraints = critical, CA:FALSE
#   keyUsage = critical, digitalSignature, keyEncipherment
#   extendedKeyUsage = serverAuth
#   subjectAltName = DNS:host.local

# TLS-Verbindungen pruefen
openssl s_client -connect host:443 -tls1_3
openssl x509 -in server.crt -noout -dates -subject -ext
    subjectAltName
```

Hashcat-Spickzettel

Spickzettel: Hashcat-Grundlagen (vertieft im Hashcat-Handbuch)

```
hashcat -b -m <modus>                          # Benchmark eines
    Modus
hashcat -a 0 -m <modus> hash.txt wortliste.txt  #
    Wörterbuchangriff
hashcat -a 3 -m <modus> hash.txt ?u?l?l?l?l?d?d?d?d  #
    Maskenangriff
hashcat --example-hashes -m <modus>             # Beispielhash für
    ein Format
```

John-the-Ripper-Spickzettel

Spickzettel: John the Ripper (vertieft im John-the-Ripper-Handbuch)

```
john --list=formats                # unterstützte Formate
    anzeigen
john --format=<format> hash.txt    # Angriff gegen eine Hashdatei
    starten
john --show hash.txt               # bereits gefundene Passwörter
    anzeigen
```

Linux-Befehle dieses Buches

Spickzettel: Linux-Bordmittel

```
sha256sum datei            # SHA-256 einer Datei berechnen
echo -n "text" | sha256sum # SHA-256 einer Zeichenkette berechnen
mkpasswd -m bcrypt "text"  # bcrypt-Hash erzeugen
ssh-keygen -t ed25519      # modernes SSH-Schlüsselpaar erzeugen
diff datei1 datei2          # zwei Dateien auf Unterschiede prüfen
xxd datei                   # Datei als Hexdump anzeigen
```

Windows-PowerShell-Befehle dieses Buches

Spickzettel: PowerShell-Bordmittel

```
Get-FileHash -Algorithm SHA256 -Path datei   # SHA-256 einer Datei
$bytes = [System.Text.Encoding]::UTF8.GetBytes("text")
$stream = [System.IO.MemoryStream]::new($bytes)
Get-FileHash -InputStream $stream -Algorithm SHA256  # SHA-256
    einer Zeichenkette
$stream.Dispose()
```

Wireshark-Filter für TLS

Spickzettel: Wireshark-Display-Filter (vertieft im Wireshark-Handbuch)

```
tls                     # gesamten TLS-Verkehr anzeigen
tls.handshake            # nur Handshake-Pakete anzeigen
tls.handshake.type == 1  # nur ClientHello-Pakete anzeigen
tls.handshake.type == 2  # nur ServerHello-Pakete anzeigen
```

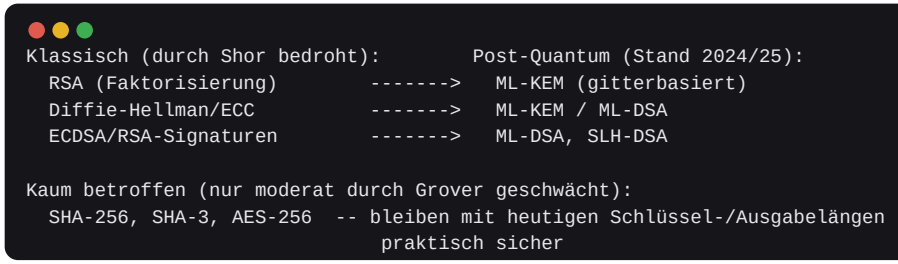
Übersicht kryptographischer Algorithmen

Kategorie	Veraltet/unsicher	Aktueller Standard
Hashfunktion	MD5, SHA-1	SHA-256, SHA-512, SHA-3
Passwort-Hash	unsalted MD5/SHA	Argon2id, bcrypt

Kategorie	Veraltet/unsicher	Aktueller Standard
Symmetrische Verschlüsselung	DES, RC4	AES-128/256-GCM
Asymmetrische Verschlüsselung	RSA unter 2048 Bit	RSA 2048+, ECC
Transportprotokoll	SSL, TLS 1.0/1.1	TLS 1.2, TLS 1.3
Post-Quantum	– (noch kein etablierter Altstandard)	ML-KEM, ML-DSA, SLH-DSA

Post-Quantum-Cryptography im Überblick

Diagramm A.1: Klassische gegenüber Post-Quantum-Verfahren



Glossar

AES – Advanced Encryption Standard, der heute gängige symmetrische Verschlüsselungsalgorithmus. **Authentizität** – die Bestätigung, dass eine Information tatsächlich von der behaupteten Quelle stammt. **Avalanche-Effekt** – die Eigenschaft, dass eine minimale Eingabeänderung den gesamten Hashwert verändert. **CA** – Certificate Authority, eine Zertifizierungsstelle innerhalb einer PKI. **Digitale Signatur** – ein eigenständiges kryptographisches Verfahren, das mit dem Private Key erzeugt und mit dem Public Key geprüft wird und Integrität sowie Authentizität bestätigt; keine einfache Umkehrung von Verschlüsselung. **ECC** – Elliptic Curve Cryptography, Oberbegriff für Verfahren auf Basis elliptischer Kurven; konkrete Einsatzzwecke sind ECDH (Schlüsselaustausch) und ECDSA/Ed25519 (Signaturen). **ECDH** – Elliptic Curve Diffie-Hellman,

ECC-Variante des Diffie-Hellman-Schlüsselaustauschs; X25519 ist eine verbreitete konkrete Ausprägung. **Ed25519** – ein auf Edwards-Kurven basierendes digitales Signaturverfahren, kein Verfahren zur Verschlüsselung oder zum Schlüsselaustausch. **Forward Secrecy** – die Eigenschaft, dass ein gestohlener Langzeitschlüssel keine vergangenen Sitzungen entschlüsseln kann; gilt bei TLS 1.3 für den üblichen (EC)DHE-Schlüsselaustausch, nicht für reine PSK-Wiederaufnahme oder 0-RTT-Daten. **Hashfunktion** – eine Funktion, die aus beliebig langer Eingabe einen deterministischen Wert fester Länge erzeugt. **HSM** – Hardware Security Module, spezialisierte Hardware zur sicheren Verwahrung privater Schlüssel. **Integrität** – die Eigenschaft, dass eine Information unverändert geblieben ist oder Änderungen erkennbar sind. **Kerckhoffs' Prinzip** – die Sicherheit eines Verfahrens soll allein vom geheimen Schlüssel abhängen, nicht von der Geheimhaltung des Verfahrens. **ML-KEM** – Key-Encapsulation-Mechanismus nach FIPS 203, gitterbasierter Post-Quantum-Ersatz für RSA- oder ECC-basierten Schlüsselaustausch. **Nonce** – ein nur einmal verwendeter Wert, der Wiederholungsmuster in verschlüsselten Daten verhindert. **PKI** – Public Key Infrastructure, das Gesamtsystem aus Zertifizierungsstellen und Zertifikaten. **Salt** – ein zufälliger, pro Konto einzigartiger Wert, der vor dem Hashen an ein Passwort angehängt wird. **TLS** – Transport Layer Security, das heutige Standardprotokoll zur Verschlüsselung von Netzwerkverbindungen. **Verbindlichkeit** – die Eigenschaft, dass ein Urheber seine Urheberschaft nicht nachträglich abstreiten kann; die technische Signaturprüfung belegt dafür nur, dass Daten und Schlüsselpaar zusammenpassen, nicht automatisch die rechtliche Wirkung oder welche Person den Schlüssel tatsächlich bedient hat. **X.509** – der Standard für den Aufbau digitaler Zertifikate.

Weiterführende Ressourcen

Die offizielle OpenSSL-Dokumentation (openssl.org), die NIST-Veröffentlichungen zu FIPS 180, 202, 203, 204 und 205 (nist.gov), sowie RFC 8446 (TLS 1.3) und RFC 5280 (X.509) der IETF bieten die verbindlichen, technischen Referenzen zu den in diesem Buch behandelten Verfahren. Das Hashcat-Handbuch, das John-the-Ripper-Handbuch und das Wireshark-Handbuch dieser Fachbibliothek vertiefen die in Kapitel 8, 9 und 10 nur angerissenen praktischen Werkzeuge vollständig.

Quellenverzeichnis

• National Institute of Standards and Technology: FIPS 180-4, Secure Hash Standard.

• National Institute of Standards and Technology: FIPS 202, SHA-3 Standard.

• National Institute of Standards and Technology: FIPS 203, Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM), https://csrc.nist.gov/pubs/fips/203/final.

• National Institute of Standards and Technology: FIPS 204, Module-Lattice-Based Digital Signature Standard (ML-DSA).

• National Institute of Standards and Technology: FIPS 205, Stateless Hash-Based Digital Signature Standard (SLH-DSA).

• National Institute of Standards and Technology: Post-Quantum Cryptography, Projektseite, https://www.nist.gov/pqcrypto.

• Internet Engineering Task Force: RFC 8446, The Transport Layer Security (TLS) Protocol Version 1.3, https://www.rfc-editor.org/rfc/rfc8446.html.

• Internet Engineering Task Force: RFC 5280, Internet X.509 Public Key Infrastructure Certificate and CRL Profile.

• OpenSSL Project: openssl-enc(1) Manual Page, https://docs.openssl.org/3.6/man1/openssl-enc/ (bestätigt, dass `enc` keine GCM-/CCM-AEAD-Modi unterstützt).

• OpenSSL Project: openssl-pkeyutl(1) Manual Page, https://docs.openssl.org/3.3/man1/openssl-pkeyutl/ (RSA-Padding-Optionen einschließlich OAEP).

• Microsoft Learn: Get-FileHash (PowerShell 5.1), https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.utility/get-filehash?view=powershell-5.1.

• OWASP Foundation: Password Storage Cheat Sheet.

• Python Cryptographic Authority: Dokumentation des Pakets `cryptography`, Modul `hazmat.primitives.ciphers.aead.AESGCM`.

Dieses Verzeichnis nennt die Dokumente mit Titel, Abrufquelle und, wo direkt relevant, der konkreten URL. Prüfe vor produktivem Einsatz jeweils die aktuell gültige Fassung; Versionsnummern und Ausgabedaten von Werkzeugen (OpenSSL, Hashcat, John the Ripper) ändern sich schneller als dieses Buch.

> **Gut zu wissen**
>
> Dieses Quellenverzeichnis nennt die Standarddokumente, auf denen die fachlichen Aussagen dieses Buches beruhen. Prüfe bei sicherheitskritischen, produktiven Entscheidungen zusätzlich die jeweils aktuelle Fassung der genannten Dokumente.

Verzeichnis der Befehlsbeispiele

Beispiel	Kapitel	Thema
1.1–1.2	1	Caesar-Chiffre, drei Grundbegriffe im Vergleich
2.1	2	Grundprinzip einer Hashfunktion
3.1	3	Salting beim Passwort-Hashing
4.1–4.2	4	Symmetrische Verschlüsselung, ECB gegenüber CBC
5.1–5.3	5	Asymmetrische Verschlüsselung, Diffie-Hellman, Hybridverschlüsselung
6.1	6	Signieren und Verifizieren

Beispiel	Kapitel	Thema
7.1–7.2	7	PKI-Grundidee, Zertifikatskette
8.1	8	TLS-1.3-Handshake
9.1–9.2	9	OSI-Modell, Verschlüsselungsarten im Vergleich
10.1	10	Angriffsstrategien im Vergleich
A.1	Anhang	Post-Quantum gegenüber klassischen Verfahren

Index

AES 4, 9, 14 · Argon2id 3, 10, 14 · Authentizität 1, 6, 14 · Avalanche-Effekt 2, 6, 13 · bcrypt 3, 10, 13 · Brute Force 3, 10 · CA 7, 13 · CRL 7 · CSR 7, 13 · Diffie-Hellman 5, 8 · Digitale Signatur 1, 6, 7, 14, Anhang · Downgrade-Angriff 8, 12 · ECC 5, 9, 11, Anhang · ECDH 5, 8, Anhang · Ed25519 5, 6, Anhang · Forward Secrecy 8, 9, Anhang · GCM 4, 12, 13 · Hashcat 3, 10, 13 · HSM 7 · Integrität 1, 6, 12 · John the Ripper 3, 10, 13 · Kerckhoffs' Prinzip 1, 7 · MD5 2, 10, 12 · ML-KEM 11, Anhang · ML-DSA 11 · Nonce 4, 12 · OAEP 5, 13 · OCSP 7 · PKI 5, 7, 13 · Post-Quantum 11, Anhang · Rainbow Table 3, 10 · RSA 5, 6, 11, 13 · Salt 3, 10, 13 · SAN 7, 8, 13 · SHA-256 2, 10, 13, 14 · Shors Algorithmus 11 · TLS 8, 9, 13 · Verbindlichkeit 1, 6, Anhang · WPA3 9 · X.509 7, Anhang

KRYPTOGRAPHIE VERSTÄNDLICH

Wie Mathematik Vertrauen im Netz möglich macht

Dieses eigenständige Praxishandbuch erklärt Kryptographie von Grund auf, ohne Mathematikstudium vorauszusetzen: wie Hashfunktionen und sichere Passwortspeicherung funktionieren, wie symmetrische und asymmetrische Verschlüsselung zusammenwirken, wie digitale Signaturen, Zertifikate und TLS Vertrauen im Netz technisch herstellen, und wie Hashcat und John the Ripper sich kryptographisch einordnen lassen.

DAS ERWARTET DICH

- Hashfunktionen, Salt und sichere Passwortspeicherung mit Argon2id und bcrypt
- AES, RSA, ECC, Diffie-Hellman und digitale Signaturen praktisch erklärt
- PKI, X.509-Zertifikate und der vollständige TLS-1.3-Handshake
- Hashcat und John the Ripper kryptographisch eingeordnet, nicht nur bedient
- Post-Quantum-Kryptographie, typische Sicherheitsfehler und ein vollständiges Praxisprojekt

Für wen?

IT-Einsteiger, angehende Fachinformatiker für Systemintegration, System- und Netzwerkadministratoren sowie Ethical Hacker und Blue-Team-Analysten, die Hashcat und John the Ripper auch kryptographisch verstehen wollen.

Praxishandbuch · Autor: Eugen Becker · Ausgabe November 2026

Hinweis: Keine Rechts- oder Anlageberatung. Marken- und Produktnamen gehören ihren jeweiligen Inhabern.